

CP Teoria

Ricardo Rojas Carvajal

August 2026

Indice rapido:

- **Plantillas y utilidades**
 - **Matematicas**
 - Teoria de numeros
 - Combinatoria
 - Polinomios y transformadas
 - Algebra lineal
 - Juegos y teoria de juegos
 - **Estructuras de datos**
 - Ordenado y contenedores
 - Union-Find
 - Arboles de segmentos
 - Otras
 - Fenwick / BIT
 - RMQ y sparse table
 - Trees y DP en arbol
 - BST balanceado
 - **Strings**
 - Hashing
 - Patrones
 - Estructuras avanzadas
 - Palindromos
 - **Grafos**
 - Caminos minimos
 - Componentes y cortes
 - Matching
 - Basicos
 - MST
 - Flujos
 - Especiales
 - **DP**
 - **Geometria**
 - Puntos y segmentos
 - Poligonos
 - Herramientas
 - **Miscelanea**
-

1 Plantillas y utilidades

1.0.1 Plantilla `main` + fast I/O

Idea intuitiva. Por defecto `cin/cout` de C++ sincroniza con la librería C (`stdio`) para que `printf` y `cin` puedan mezclarse. Esa sincronización hace que cada operación de I/O pase por un lock y vuelque buffers, lo cual es **muy lento** cuando se leen miles de números. Desactivar la sincronización con `ios_base::sync_with_stdio(false)` y desatar `cin` de `cout` con `cin.tie(0)` deja a `cin` usando su propio buffer, dando una velocidad comparable (y a veces mejor) que `scanf` pero con la comodidad de los operadores `>>/<<`. El `\n` se prefiere sobre `endl` porque este último además hace flush del buffer, anulando parte de la ganancia.

Propiedades clave (invariantes).

- Tras `sync_with_stdio(false)`, **no mezclar** `cin/cout` con `printf/scanf`: el orden de salida queda indefinido.
- Tras `cin.tie(0)`, cada `cout << x` no fuerza un flush automático previo del buffer de `cin` (útil cuando se hace `cout << "resultado: " << ans << "\n";` justo después de leer con `cin`).

Complejidad.

- Sin fast I/O: $\sim 10\times$ más lento en problemas con 10^5 o más entradas.
- Con fast I/O: tiempo lineal en el tamaño de la entrada, suficiente para casi todos los contests.

Donde tocar para modificar.

- Si el problema lee strings con espacios, cambiar `cin >> s` por `getline(cin, s)` (y recordar consumir el `\n` residual con `cin.ignore()` si se mezcla con `cin >>`).
- Si los números no entran en `int` (por ejemplo $N \leq 10^{18}$), cambiar el tipo de `n`, `x` y de los contenedores a `long long`.
- Si necesitas **imprimir con precisión fija** (decimales), agregar al inicio: `cout << fixed << setprecision(10);` (incluir `<iomanip>`).

Trampas y errores frecuentes.

- Mezclar `scanf` y `cin` después de `sync_with_stdio(false)` produce salidas raras; usar solo uno.
- Usar `endl` en lugar de `"\n"` puede ser $\sim 5\times$ más lento si se imprime muchas líneas.
- En problemas interactivos, a veces es necesario **volver a atar** `cin` y forzar flush; usar `cout << flush;` o `endl` solo ahí.

Código. Ver `cpp.tex`, sección *Plantilla main + fast I/O*.

1.0.2 Random / mt19937

Idea intuitiva. `mt19937_64` es un generador de números pseudoaleatorios (Mersenne Twister) con periodo $2^{19937} - 1$. A diferencia del `rand()` de C, tiene una distribución razonablemente uniforme y se puede **seedear** explícitamente, lo que permite reproducir exactamente la misma secuencia. En contests se usa para: barajar vectores en algoritmos randomized (quicksort, kruskal randomizado), generar casos de prueba, y romper empates con hash aleatorio.

Propiedades clave (invariantes).

- Periodo gigantesco: la probabilidad de repetir es despreciable en cualquier contest.
- Produce 64 bits por llamada; si necesitas `int`, tomar el valor módulo el rango deseado.
- `uniform_int_distribution<int>(lo, hi)` es **inclusivo en ambos extremos**; `uniform_real_distribution<double>` es **semi-abierto** $[lo, hi)$.

Complejidad.

- $O(1)$ por llamada. Construir el `rng` con seed de `chrono` es $O(1)$.

Donde tocar para modificar.

- **Reproducibilidad:** cambiar el seed de `chrono::steady_clock::now()`... por un literal (`42ULL`, etc.) si quieres exactamente la misma secuencia en cada corrida.
- **Tipo:** usar `mt19937` (32 bits) en lugar de `mt19937_64` si solo necesitas `int`; es ligeramente mas rapido pero no se nota en la practica.
- `randInt(lo, hi)` con `lo > hi` da comportamiento indefinido; aniaade un `assert` si te preocupa.
- `randPick(v)` hace `v.size() - 1`; si `v` esta vacio hay UB. Aniaadir check.

Trampas y errores frecuentes.

- En Windows (MSVC) el `uniform_int_distribution` puede tener un bug conocido con distribuciones grandes; usar `mt19937_64` lo evita.
- Llamar a `rand() % n` es **sesgado** si `RAND_MAX + 1` no es multiplo de n . `uniform_int_distribution` no tiene ese problema.

Codigo. Ver `cpp.tex`, seccion *Random / mt19937*.

1.0.3 BigInt (add + multiply, base 1e9)

Idea intuitiva. Un `long long` soporta hasta $\approx 9 \times 10^{18}$. Si el problema requiere operar con numeros de cientos o miles de digitos (factoriales grandes, sumas de Catalan, etc.), hace falta una representacion propia. La eleccion mas simple es almacenar los digitos en **base** 10^9 dentro de un `vector<int>` (least significant first): asi cada `int` guarda un “bloque” de 9 digitos, y las operaciones aritmeticas se reducen a lo que aprendimos en la escuela, pero con bloques en vez de digitos. La suma es lineal en el numero de bloques; la multiplicacion por un `int` (no por otro `BigInt`) tambien lineal.

Propiedades clave (invariantes).

- Base 10^9 asegura que la suma de dos bloques ($\max 2 \cdot 10^9 - 2$) entra en `long long` y el carry es a lo sumo 1.
- `d` siempre tiene el least significant block primero (indice 0).
- No hay ceros a la izquierda en `d`: si el numero es 0, `d` esta vacio.
- `neg` indica signo; las operaciones asumen mismo signo.

Complejidad.

- `+=`: $O(N)$ donde $N = \max(d.size(), other.d.size())$.
- `*=` (por `int`): $O(N)$.
- `toString`: $O(N)$.
- Multiplicacion entre dos `BigInt`: $O(N^2)$ si se hace con el metodo clasico; **no implementado** en este snippet (habria que aniaadir un `*=` que itera bloques o usar FFT para $O(N \log N)$).

Donde tocar para modificar.

- **Aniaadir multiplicacion entre BigInt:** aniaadir `operator*=(const BigInt&)` o `operator*`. Para $N \sim 1000$ bloques la version $O(N^2)$ sirve; para mas, usar FFT sobre coeficientes `long long`.
- **Aniaadir resta con signo:** complica el invariante “mismo signo”; mejor aniaadir un `sign()` interno y hacer la resta en valor absoluto, luego ajustar el signo.
- **Cambiar base:** usar base 10^4 o 10^{18} acelera la multiplicacion pero requiere mas cuidado con overflow al multiplicar (usar `_int128`).
- **Imprimir negativo:** ya hay soporte parcial en `toString()` con el flag `neg`; falta activarlo en las operaciones.

Trampas y errores frecuentes.

- Multiplicar dos bloques en base 10^9 puede dar hasta $\approx 10^{18}$ que entra en `long long` pero **no** en `int`. Usar siempre `long long` o `__int128` para el producto.
- `toString` usa `%09d` para rellenar con ceros a la izquierda; si la base cambia, hay que ajustar el formato.
- El snippet NO tiene operador `<`, `>`, ni division; si los necesitas, aniadelos con cuidado.

Codigo. Ver `cpp.tex`, seccion *BigInt (add + multiply, base 1e9)*.

1.0.4 Debug pretty-printer

Idea intuitiva. Imprimir `vector<int>{1,2,3}` con `cout << v` da una direccion de memoria, no el contenido. Hay que iterar manualmente. Los macros `DBG(x)` y `DBGV(x)` envuelven esa iteracion y muestran, ademas del valor, el **nombre de la variable** (`#x` lo convierte en string en tiempo de preprocesamiento). Asi un `DBG(ans)` imprime `ans = 42`, y un `DBGV(arr)` imprime `arr = [1, 2, 3]`. Esto acelera muchisimo el debug: con solo aniadir una linea ves el estado de una variable.

Propiedades clave (invariantes).

- `#x` en una macro **siempre** toma el nombre textual tal cual lo escribiste; si pasas `arr[i]`, imprime `arr[i] = ...`, no el nombre del array.
- `DBGV` solo funciona para `vector<T>`; para `set`, `map`, etc. aniadir overloads especificos.
- Los macros escriben a `cerr` (no `cout`), asi no contaminan la salida del problema.

Complejidad.

- $O(N)$ por impresion donde N = tamano del contenedor.
- El overhead del macro es despreciable.

Donde tocar para modificar.

- **Aniadir tipos** (`set`, `map`, `pair` de vectores, etc.): duplicar la plantilla de `vecToStr` para cada tipo y crear un `DBG##` correspondiente.
- **Volcar a archivo:** cambiar `cerr` por un `ofstream` abierto al inicio.
- **Quitar del envio:** las macros se activan solo donde estan definidas; antes de enviar al juez, **borrar las lineas con** `DBG/DBGV` o aniadir `#undef` despues del bloque (como hace el snippet).
- **Debug condicional:** aniadir `#ifdef LOCAL` antes de las macros y compilar con `-DLOCAL` solo en local.

Trampas y errores frecuentes.

- Si el output del juez es estricto (case-sensitive, sin espacios extra), no debe quedar ningun `cerr` activo.
- En recursion profunda, los `DBG` pueden explotar el stack de `cerr` (raro, pero pasa). Usar `\n` (no `endl`) para evitar flush.
- Macros con argumentos multiples (`DBG(a, b)`) requieren variadic macros (`#define DBG(...)`) en C++11.

Codigo. Ver `cpp.tex`, seccion *Debug pretty-printer*.

1.0.5 Coordinate compression

Idea intuitiva. Muchas estructuras (`BIT`, `segment tree`, `sparse table`) indexan con enteros pequenos en $[0, N)$. Si el problema da valores hasta 10^9 , no puedes usarlos como indices directamente. La “compresion de coordenadas” reemplaza cada valor por su **rango** en el conjunto ordenado de valores distintos: el menor valor es 0, el siguiente es 1, etc. Asi, si los datos son $\{10^9, 5, 100, 42\}$, la compresion da $\{3, 0, 2, 1\}$. Esto preserva el orden y las relaciones de igualdad/desigualdad, pero con indices pequenos y densos.

Propiedades clave (invariantes).

- Si $a < b$ en los datos originales, entonces `compress(a) < compress(b)`.
- Si $a = b$, `compress(a) = compress(b)`.
- Los indices resultantes estan en $[0, K)$ donde K = numero de valores distintos ($\leq N$).
- `compressWithMap` ademas devuelve el vector inverso: en la posicion i esta el valor original que mapea a i .

Complejidad.

- $O(N \log N)$: el sort domina. Las busquedas binarias son $O(\log N)$ cada una.
- Memoria $O(N)$.

Donde tocar para modificar.

- **Preservar duplicados**: si necesitas mantener la posicion original de cada valor ademas del rango, anadir un `struct pair<long long, int>` con el valor y el indice original antes de ordenar.
- **Compression online (con inserciones)**: si llegan valores en stream y necesitas comprimirlos, usar una `policy-based data structure` (ordered set) o `coordinate_compression` con insercion en `vector` ordenado en $O(\log N)$ por insercion.
- **Mapear a $[1, K]$ en vez de $[0, K)$** : anadir `+ 1` al final (util para BIT que usa indices 1-based).
- **3D compression**: para problemas donde hay que comprimir dos coordenadas a la vez (tipico de “rectangulos disjuntos”), comprimir cada eje por separado y emparejar por valor original.

Trampas y errores frecuentes.

- No olvides `sorted.erase(unique(...), ...)`; si lo olvidas, valores repetidos daran rangos duplicados.
- Si los datos tienen signo (incluyen negativos), `lower_bound` sigue funcionando; no hace falta un truco extra.
- Compression **no preserva distancias**: si la diferencia absoluta importa (por ejemplo, $|a - b|$), el rango no sirve; necesitas el valor original.

Codigo. Ver `cpp.tex`, seccion *Coordinate compression*.

2 Matematicas

2.1 Teoria de numeros

2.1.1 Extended Euclidean Algorithm

Idea intuitiva. Dados dos enteros a y b no ambos cero, existen enteros x, y tales que $ax + by = \gcd(a, b)$. El algoritmo extendido de Euclides los construye **mientras** calcula el gcd, mediante recursion: si (x_1, y_1) resuelve $b \cdot x_1 + (a \bmod b) \cdot y_1 = \gcd$, entonces $x = y_1$ e $y = x_1 - y_1 \cdot \lfloor a/b \rfloor$ resuelve $ax + by = \gcd$.

Propiedades clave (invariantes).

- Siempre termina (Euclides baja el segundo argumento).
- x puede ser negativo.
- Si $\gcd(a, b) = 1$, entonces x es el **inverso modular** de a modulo b .

Complejidad.

- $O(\log \min(a, b))$ operaciones.

Donde tocar para modificar.

- **Inversa modular:** la linea `x = y1; y = x1 - y1 * (a/b);` resuelve el caso recursivo. Para usar como inversa modular con mod primo, despues ajustar al rango $[0, \text{mod})$.
- **Cuidado con negativos:** si b es negativo, ajustar la division entera.
- Si el problema requiere **muchas** inversas del mismo mod, conviene precomputar con el algoritmo de Fermat $(a^{p-2} \bmod p)$ en lugar de llamar a `extgcd` por cada una.

Trampas y errores frecuentes.

- Si los numeros exceden `int`, cambiar a `long long`. El snippet retorna `int`; insuficiente para $a > 2^{31}$.

Codigo. Ver `cpp.tex`, seccion *Extended Euclidean Algorithm*.

2.1.2 Formulas utiles

Idea intuitiva. Mini-referencia de identidades clasicas: numero de divisores, suma de divisores, formula de Euler ($a^b \equiv a^{b \bmod \varphi(n)} \pmod{n}$ cuando $\gcd(a, n) = 1$), e inclusion-exclusion. **No hay codigo ejecutable;** solo las formulas y, opcionalmente, comentarios sobre cuando aplicarlas.

Propiedades clave (invariantes).

- Numero de divisores: si $n = \prod p_i^{e_i}$, entonces $d(n) = \prod (e_i + 1)$.
- Suma de divisores: $\sigma(n) = \prod \frac{p_i^{e_i+1} - 1}{p_i - 1}$.
- Producto de divisores: $n^{d(n)/2}$.
- Inclusion-exclusion: $|\bigcup A_i| = \sum_{\emptyset \neq I} (-1)^{|I|+1} |\bigcap_{i \in I} A_i|$.

Complejidad. No aplica para este modulo (es solo referencia de formulas).

Donde tocar para modificar. Este modulo **no tiene codigo**; es una seccion de formulas. Si quieres anadir una formula nueva (por ejemplo, suma de primos hasta n via criba), agregar el `\subsubsection` correspondiente en otra parte del notebook.

Trampas y errores frecuentes.

- La suma de primos no tiene formula cerrada simple; necesita criba.
- $\varphi(n)$ solo simplifica el exponente cuando $\gcd(a, n) = 1$; en caso contrario, hay que factorizar a mano.

Codigo. Ver `cpp.tex`, seccion *Formulas utiles*.

2.1.3 Criba de Eratostenes

Idea intuitiva. Para encontrar todos los primos hasta N , marcaremos como compuestos los multiplos de cada primo a partir de p^2 . Como el primo mas pequeno es 2, empezar en 4 evita marcar de mas. Es $O(N \log \log N)$ porque la cantidad de primos hasta N es $\sim N / \ln N$ y la mayoria tiene pocos multiplos pequenos.

Propiedades clave (invariantes).

- `is_composite[i]` es true sii i no es primo (o $i \leq 1$).
- `primes` contiene todos los primos en orden ascendente.
- Se itera solo hasta $\sqrt{N}$ en el bucle exterior porque cualquier compuesto $c \leq N$ tiene un factor primo $\leq \sqrt{N}$.

Complejidad.

- Tiempo $O(N \log \log N)$, memoria $O(N)$.

Donde tocar para modificar.

- **Obtener factorizacion de cada numero:** anadir un array `lp[i]` (lowest prime factor) que se llena junto al sieve. Es $O(N)$ si se hace en orden creciente de i .
- **Criba hasta $N = 10^7$:** este snippet con `vector<bool>` (que es bitset-packed) alcanza. Para $N = 10^8$ o mas, conviene criba segmentada (no incluida).
- **Sumar primos:** anadir `long long sumPrimes;` que acumule en el bucle final.

Trampas y errores frecuentes.

- `vector<bool>` es proxy; si necesitas pasarlo a C-style API, copiar a `vector<char>`.
- `is_composite[0]` e `is_composite[1]` deben inicializarse a `true`.

Codigo. Ver `cpp.tex`, seccion *Criba de Eratostenes*.

2.1.4 Criba lineal

Idea intuitiva. La criba de Eratostenes marca cada compuesto varias veces (por ejemplo 12 se marca por 2, por 3 y por 4). La criba lineal asegura que **cada compuesto se marca exactamente una vez**, lo cual da $O(N)$ total. La idea: para cada i , iterar sobre los primos p en orden creciente y poner `lp[i · p] = p`, **cortando** el bucle cuando $p = lp[i]$ (asi $i · p$ queda con su factor primo mas chico como p).

Propiedades clave (invariantes).

- `lp[i]` = menor factor primo de i (con `lp[0] = lp[1] = 0`).
- Cada i tiene asignado exactamente un primo que “lo cierra”.
- Recorriendo i en orden creciente, cuando entramos a i , todos sus primos menores ya estan en `primes`.

Complejidad.

- Tiempo $O(N)$ exacto, memoria $O(N)$.

Donde tocar para modificar.

- **Factorizacion de un numero n :** usando `lp`, iterar `while (n > 1) { p = lp[n]; ...; n /= p; }` en $O(\log n)$.
- **Cantidad de divisores para todos los $i \leq N$:** anadir `cntDiv[i] = cntDiv[i/lp[i]] + 1` durante la criba.
- **Suma de divisores o totient por i :** igual, precomputar en el mismo pase.

Trampas y errores frecuentes.

- La condicion `if (p == lp[i]) break;` es **crucial**; sin ella la criba vuelve a ser $O(N \log \log N)$.
- Overflow: chequear `1LL * i * p > n` antes de asignar.

Codigo. Ver `cpp.tex`, seccion *Criba lineal*.

2.1.5 Miller-Rabin primality

Idea intuitiva. Para n impar, escribir $n - 1 = 2^s \cdot d$ con d impar. Si n es primo, entonces para **cualquier** a con $1 < a < n - 1$ se cumple que $a^d \equiv 1 \pmod{n}$ o $a^{2^r \cdot d} \equiv -1 \pmod{n}$ para algun $r < s$. El test verifica esto con varios a (testigos). Si ningun testigo delata a n , es primo. Con un conjunto fijo de testigos, es **determinista** para $n < 2^{64}$.

Propiedades clave (invariantes).

- Probabilístico si los testigos son aleatorios; **determinista** con el set del snippet para $u64$.
- Falla para $n < 2$ (no es primo).
- Bases: $\{2, 325, 9375, 28178, 450775, 9780504, 1795265022\}$ cubren todos los $u64$.

Complejidad.

- $O(\log^3 n)$ por la exponenciación modular, pero con constante pequeña. Para $u64$, unos pocos microsegundos.

Donde tocar para modificar.

- **Cambiar el set de testigos:** si necesitas testear números $> 2^{64}$, añadir más testigos o usar BPSW.
- **Devolver un certificado:** añadir la secuencia de cuadrados que llevo al fallo para debugging.
- **u128:** el snippet usa `__uint128_t` para multiplicar mod n sin overflow. En GCC/clang funciona; en MSVC añadir `#include <boost/multiprecision/cpp_int.hpp>`.

Trampas y errores frecuentes.

- Multiplicar $u64 * u64$ puede overflow; **siempre** usar `u128`.
- Si n es par, el test responde rápido (lo chequea con `n % p == 0`).
- Cuidado con bases que sean múltiplos de n ; el snippet las saltea con `if (a % n == 0) continue;`.

Código. Ver `cpp.tex`, sección *Miller-Rabin primality*.

2.1.6 Pollard's Rho + factorización

Idea intuitiva. Miller-Rabin detecta primalidad pero no factoriza. Pollard's Rho busca un **factor no trivial** de n usando una secuencia pseudoaleatoria $x_{k+1} = x_k^2 + c \pmod n$ y aplicando la **paradoja del cumpleaños**: si la secuencia cae en un ciclo módulo un factor primo p de n , entonces $\gcd(|x_i - x_j|, n)$ será p para algún i, j . Esto es esperado $O(n^{1/4})$ por factor.

Propiedades clave (invariantes).

- Cada llamada a `pollard(n)` devuelve **un** factor (no necesariamente primo); se llama recursivamente.
- Si `millerRabin(n)` retorna `true`, n es primo y se añade directo.
- El factor final sale **ordenado** porque el snippet ordena al final.

Complejidad.

- Esperado $O(n^{1/4})$ por factor. Práctico para n hasta 10^{18} .

Donde tocar para modificar.

- **Mejorar la constante:** añadir el algoritmo de Brent (variante del Floyd para el ciclo).
- **Determinismo:** si la corrida con c falla, reintentar con otro c aleatorio (ya lo hace el snippet con `while(true)`).
- **Salida con multiplicidad:** el snippet devuelve la lista con repeticiones (porque `factorRec` empuja n cuando es primo y también n/d). Si solo quieres factores distintos, añadir un `set` o deduplicar al final.

Trampas y errores frecuentes.

- Si n es potencia de un primo pequeño (ej. 2^k), Pollard puede tardar. Considerar chequeo previo de potencia.
- Si n es primo, llamar a `pollard(n)` causaría loop infinito porque `d == n`; el snippet tiene la guarda `if (millerRabin(n)) return`.

Codigo. Ver `cpp.tex`, seccion *Pollard's Rho + factorizacion*.

2.1.7 Modular exponentiation

Idea intuitiva. Calcular $a^e \bmod m$ en $O(\log e)$ mediante **exponenciacion binaria** (squaring). La idea: si e es par, $a^e = (a^{e/2})^2$; si es impar, $a^e = a \cdot (a^{(e-1)/2})^2$. Asi, en cada paso, el exponente se reduce a la mitad y se hace una multiplicacion modular.

Propiedades clave (invariantes).

- $a \% m$ se hace al inicio (en caso de que a sea negativo, se ajusta $+= \text{mod}$).
- Multiplicacion: para evitar overflow, usar `__int128` o `long long` solo si $m < 2^{32}$ (porque $a \cdot b < m^2 < 2^{64}$).
- Resultado siempre en $[0, m)$.

Complejidad.

- $O(\log e)$ multiplicaciones modulares.

Donde tocar para modificar.

- **Modulo de 10^{18} :** anadir `__int128` a las multiplicaciones (ya hecho si el snippet usa `u128`; si no, anadir manualmente).
- **Exponentes muy grandes** ($e > 10^{18}$): el bucle sigue funcionando pero no entra; anadir modularizacion del exponente via Euler si $\gcd(a, m) = 1$ y quieres reducir e primero.
- **Pow matrix/Poly:** la misma estructura sirve; solo cambia el tipo de la base y la operacion (mul de matrices, mul de polinomios).

Trampas y errores frecuentes.

- Si m es 1, el resultado siempre es 0; anadir check si te molesta.
- Overflow en $a * a \% \text{mod}$ si a y mod son `long long` grandes.

Codigo. Ver `cpp.tex`, seccion *Modular exponentiation*.

2.1.8 Modular inverse

Idea intuitiva. La **inversa** de a modulo m es un b tal que $a \cdot b \equiv 1 \pmod{m}$. Si m es primo y $\gcd(a, m) = 1$, entonces $b = a^{m-2} \pmod{m}$ (por el **teorema pequeno de Fermat**). Si m no es primo, se usa el algoritmo extendido de Euclides, que resuelve $ax + my = \gcd(a, m)$.

Propiedades clave (invariantes).

- La inversa existe sii $\gcd(a, m) = 1$.
- Para mod primo, $\text{modInv}(a, \text{mod}) = \text{modpow}(a, \text{mod} - 2, \text{mod})$.
- `modInvGeneral` retorna -1 si no hay inversa.

Complejidad.

- $O(\log \text{mod})$ en ambos casos.

Donde tocar para modificar.

- **Precomputar muchas inversas** (ej. para todos los $1..N$): precomputar en $O(N)$ con la recurrencia `inv[i] = -(mod/i) * inv[mod % i] % mod` y ajustar a positivo.
- **Cambiar mod:** si modificas el `MOD` global, asegurate de que sea primo (sino, usar `extgcd`).

Trampas y errores frecuentes.

- Si $\text{MOD} = 1$, $a^{\text{MOD}-2} = a^{-1}$ no esta definido; anadir check.
- Si $a = 0$, no hay inversa (excepto en mod trivial).
- El snippet usa `modInv(a, mod)` que asume mod primo por construccion.

Codigo. Ver `cpp.tex`, seccion *Modular inverse*.

2.1.9 Chinese Remainder Theorem

Idea intuitiva. Dado un sistema $x \equiv r_i \pmod{m_i}$ con los m_i **coprimos dos a dos**, hay **una** solucion modulo $M = \prod m_i$. Se construye incrementalmente: combinando $x \equiv r_1 \pmod{m_1}$ con $x \equiv r_2 \pmod{m_2}$, la solucion es $x = r_1 + m_1 \cdot t$ donde $t \equiv (r_2 - r_1) \cdot m_1^{-1} \pmod{m_2}$. Iterar con cada m_i .

Propiedades clave (invariantes).

- El snippet asume m_i coprimos dos a dos; **no** maneja el caso general.
- Solucion unica modulo M .
- Si los modulos **no** son coprimos y $\gcd(m_i, m_j) \nmid (r_i - r_j)$, no hay solucion; si divide, se reduce el sistema.

Complejidad.

- $O(N \log M)$ con N el numero de ecuaciones.

Donde tocar para modificar.

- **Modulos no coprimos:** anadir el test $\gcd(m_i, m_j)$ en cada paso y devolver error o reducir.
- **Devolver todas las soluciones:** si el problema quiere todas las x en $[0, L]$, ajustar al final con un `if (x > L) return -1;`.
- **Garner:** para evitar overflow con M muy grande, usar el algoritmo de Garner (mas estable numericamente).

Trampas y errores frecuentes.

- Overflow al multiplicar `M * t` si los m_i son grandes; usar `__int128` o reducir modulo M en cada paso (ya lo hace el snippet).

Codigo. Ver `cpp.tex`, seccion *Chinese Remainder Theorem*.

2.1.10 Euler totient precompute

Idea intuitiva. La funcion $\varphi(n)$ cuenta cuantos enteros en $[1, n]$ son coprimos con n . Hay una formula: $\varphi(n) = n \cdot \prod_{p|n} (1 - 1/p)$. Para precomputar $\varphi(i)$ para todo $i \leq N$ en $O(N \log \log N)$, se inicializa $\varphi(i) = i$ y para cada primo p se hace $\varphi(j) -= \varphi(j)/p$ para todos los multiplos j de p .

Propiedades clave (invariantes).

- $\varphi(1) = 1$ por convencion (aunque coprimos con 1 es solo $\{1\}$).
- φ es **multiplicativa**: si $\gcd(a, b) = 1$, $\varphi(ab) = \varphi(a)\varphi(b)$.
- **Identidad de Euler**: si $\gcd(a, n) = 1$, entonces $a^{\varphi(n)} \equiv 1 \pmod{n}$.

Complejidad.

- Precompute $O(N \log \log N)$, consulta $O(1)$. Single $\varphi(n)$ con factorizacion es $O(\sqrt{n})$.

Donde tocar para modificar.

- **Single phi via factorizacion:** ya hay una funcion `phiSingle(n)`. Para n hasta 10^{12} factoriza en $\leq 10^6$ pasos.
- **Aplicar a inclusion-exclusion:** si el problema requiere $\sum_{i=1}^N \gcd(i, n)$, se puede usar $\sum_{d|n} d \cdot \varphi(n/d)$.

Trampas y errores frecuentes.

- Olvidar inicializar $\varphi(0) = 0$.
- Si llamas a `phiSingle` sobre 0, devuelve 0 (no hay coprimos con 0 en el sentido usual).

Codigo. Ver `cpp.tex`, seccion *Euler totient precompute*.

2.1.11 Mobius precompute

Idea intuitiva. La funcion $\mu(n)$ (“Mobius”) vale 1 si n es producto de un numero par de primos distintos, -1 si es impar, 0 si tiene un primo al cuadrado. Aparece en inclusion-exclusion: si $f(n) = \sum_{d|n} g(d)$, entonces $g(n) = \sum_{d|n} \mu(n/d) f(d)$ (**inversion de Mobius**). Se precomputa junto a la criba lineal.

Propiedades clave (invariantes).

- $\sum_{d|n} \mu(d) = [n = 1]$ (funcion indicador de 1).
- Si $\gcd(a, b) = 1$, $\mu(ab) = \mu(a)\mu(b)$.

Complejidad.

- Precompute $O(N)$ con la criba lineal. Consulta $O(1)$.

Donde tocar para modificar.

- **Suma de gcd:** $\sum_{i=1}^N \gcd(i, n) = \sum_{d|n} d \cdot \varphi(n/d)$, alternativa con Mobius: $\sum_{d|n} \mu(d) \cdot S(\lfloor n/d \rfloor)$ donde $S(x) = \sum_{k \leq x} k$.
- **Mobius en inclusion-exclusion:** el snippet es solo para precompute; usarlo combinado con divisor enumeration en queries.

Trampas y errores frecuentes.

- $\mu(0) = 0$ (por definicion, 0 no es producto de primos distintos).
- El snippet inicializa `mu[0] = 0`, `mu[1] = 1` por separado.

Codigo. Ver `cpp.tex`, seccion *Mobius precompute*.

2.1.12 Stern-Brocot (best fraction in interval)

Idea intuitiva. Stern-Brocot es un arbol binario de fracciones irreducibles. Cada nodo es el **mediante** de sus “padres” en el arbol: si tienes a/b y c/d , el mediantes es $(a+c)/(b+d)$. Para encontrar la fraccion irreducible con denominador acotado mas cercana a un valor x , se navega el arbol decidiendo ir a izquierda o derecha segun si x es menor o mayor. La fraccion mas simple en un intervalo $(a/b, c/d)$ se obtiene iterando mediantes y quedandose con el **maximo** k tal que el mediantes sigue dentro del intervalo.

Propiedades clave (invariantes).

- Cada par de fracciones irreducibles consecutivas en Stern-Brocot tiene diferencia $1/(bd)$ en valor.
- Permite generar todas las fracciones irreducibles con un denominador acotado.

Complejidad.

- $O(\log(\max(a, b)))$ por query.

Donde tocar para modificar.

- **Acotar denominador:** añadir un parametro Q (denominador maximo) y ajustar $k = \min(k, Q/\dots)$.
- **Devolver la fraccion irreducible (no reducida):** añadir division por $\gcd(p, q)$ al final.

Trampas y errores frecuentes.

- El snippet tiene una aproximacion cruda; un mejor implementation debe iterar **exactamente** el maximo k permitido.
- Cuidado con overflow en $(p - a) / b$ si los numeros son grandes.

Codigo. Ver `cpp.tex`, seccion *Stern-Brocot (best fraction in interval)*.

2.2 Combinatoria

2.2.1 Catalan (formula)

Idea intuitiva. Los numeros de Catalan cuentan una gran variedad de estructuras (arboles binarios, parentesis balanceados, triangulaciones de poligono). La formula es $C_n = \frac{1}{n+1} \binom{2n}{n}$. La version recursiva es $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$, que da una DP en $O(n^2)$.

Propiedades clave (invariantes).

- $C_0 = C_1 = 1$.
- C_n crece rapidamente; C_{19} ya pasa el millon.
- La formula cerrada evita la DP pero requiere multiplicacion que **siempre** es exacta (la division por $i + 2$ no deja residuo).

Complejidad.

- DP: $O(N^2)$ tiempo, $O(N)$ memoria.
- Formula cerrada (iterativa): $O(N)$ tiempo, $O(1)$ memoria.

Donde tocar para modificar.

- **Catalan mod p:** añadir MOD y hacer $\% \text{ MOD}$ en cada multiplicacion. La division $/(i+2)$ no se puede hacer mod p directamente; conviene usar el inverso modular: `ans = ans * 2 * (2*i+1) % MOD * inv[i+2] % MOD;`
- **Catalan con doble precision:** `catalanLL` retorna `long long`; para $n \geq 35$ desborda.

Trampas y errores frecuentes.

- La division entera $(i+2)$ en `catalanLL` es exacta solo porque matematicamente el cociente es entero. Si cambia el orden de operaciones (por ejemplo, modular primero), la division falla.

Codigo. Ver `cpp.tex`, seccion *Catalan (formula)*.

2.2.2 Factoriales + $nCr \bmod p$

Idea intuitiva. Si p es primo, $\binom{n}{k} \bmod p$ se puede calcular en $O(1)$ con factoriales precomputados: $\binom{n}{k} = \frac{n!}{k!(n-k)!} = n! \cdot (k!)^{-1} \cdot ((n-k)!)^{-1} \pmod{p}$. Las inversas se precomputan una vez con $fact^{-1}[n] = fact[n]^{p-2}$ y se propaga hacia abajo.

Propiedades clave (invariantes).

- Precompute en $O(N)$, query en $O(1)$.
- Asume p primo; si p no es primo, la inversa modular no siempre existe.
- $\binom{n}{k} = 0$ si $k > n$.

Complejidad.

- Precompute $O(N)$, query $O(1)$.

Donde tocar para modificar.

- **n grande con p primo pequeño** ($\leq 10^6$): usar Lucas theorem.
- **Cambiar mod:** añadir nuevo MOD y regenerar `fact/invFact`.
- **Sumar $\binom{n}{k}$ sobre k :** usar DP o suma geometrica.

Trampas y errores frecuentes.

- MOD = 1 no es primo, falla en `modpow(fact[n], MOD - 2)`.
- `buildFact(0)` deja `fact = {1}`; `nCr(0, 0) = 1` OK.

Codigo. Ver `cpp.tex`, seccion *Factoriales + nCr mod p*.

2.2.3 Lucas theorem

Idea intuitiva. Para p primo y n, k grandes, $\binom{n}{k} \bmod p$ se calcula descomponiendo n y k en base p : $n = n_0 + n_1p + n_2p^2 + \dots$, $k = k_0 + k_1p + \dots$. Entonces $\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}$ (**Lucas**). Cada $\binom{n_i}{k_i}$ se calcula con el snippet anterior.

Propiedades clave (invariantes).

- Solo funciona con p primo.
- Si algun $k_i > n_i$, el coeficiente es 0.
- Reduce el problema a $\log_p(n)$ coeficientes chiquitos.

Complejidad.

- $O(\log_p n)$ multiplicaciones, cada una requiere un $\binom{n_i}{k_i}$ que es $O(1)$ con factoriales precomputados.

Donde tocar para modificar.

- **p no primo:** requiere extender Lucas a modulos compuestos (no trivial, basado en factorizacion).
- **Devolver la suma:** si necesitas $\sum_k \binom{n}{k} \bmod p$, es $2^n \bmod p$ via Fermat (cuando p es primo).

Trampas y errores frecuentes.

- `buildFact(MOD)` solo precomputa hasta MOD, no hasta `n`. Pero como cada $\binom{n_i}{k_i}$ tiene $n_i < p$, eso es suficiente.

Codigo. Ver `cpp.tex`, seccion *Lucas theorem*.

2.2.4 Stirling S2 (particiones)

Idea intuitiva. $S(n, k)$ = numero de formas de particionar n elementos en k subconjuntos no vacios. La recurrencia es $S(n, k) = S(n-1, k-1) + k \cdot S(n-1, k)$ (o pones el elemento n -esimo solo en un nuevo subconjunto, o lo metes en uno de los k existentes).

Propiedades clave (invariantes).

- $S(n, 0) = [n = 0]$.
- $S(n, n) = 1$, $S(n, 1) = 1$.
- $\sum_k S(n, k) = B_n$ (numero de Bell).

Complejidad.

- $O(N^2)$ tiempo y memoria.

Donde tocar para modificar.

- **Stirling de primera especie**: recurrencia distinta; añadir nueva tabla.
- **Memory optimization**: solo guardar la fila anterior (DP en espacio $O(N)$).
- **Output sin mod**: si el conteo es chico, retornar `long long` y omitir `% MOD`.

Trampas y errores frecuentes.

- Olvidar inicializar `S2[0][0] = 1` y el resto en 0.
- Overflow sin modulo.

Codigo. Ver `cpp.tex`, seccion *Stirling S2 (particiones)*.

2.2.5 Inclusion-exclusion over subsets

Idea intuitiva. Para contar elementos que cumplen **alguna** de n propiedades, la formula clasica de inclusion-exclusion es: $|\bigcup A_i| = \sum_{\emptyset \neq I} (-1)^{|I|+1} |\bigcap_{i \in I} A_i|$. Esto se computa iterando sobre todos los $2^n - 1$ subconjuntos no vacios y aplicando el signo correspondiente.

Propiedades clave (invariantes).

- El usuario pasa una funcion `f(mask)` que cuenta cuantos elementos cumplen **exactamente** las propiedades en `mask`.
- El signo es $(-1)^{|I|+1}$, equivalente a $+1$ si $|I|$ impar, -1 si par.
- Para $n \leq 20$ es factible ($2^{20} \approx 10^6$); para mas, usar otra tecnica (Mobius, formula cerrada).

Complejidad.

- $O(2^n \cdot \text{costo de } f)$.

Donde tocar para modificar.

- **Subset de subconjuntos** (elementos con un subconjunto de propiedades): usar bitmask para mapear cada elemento a un mask, luego contar ocurrencias por mask y sumar con signo.
- **Aniadir la interseccion**: en lugar de $|\bigcap A_i|$ exacta, aproximar o usar otra estructura.
- **Cambiar el signo**: si tu problema tiene inclusion-exclusion “inversa”, invertir el patron $\pm$.

Trampas y errores frecuentes.

- El snippet dado es solo el **esqueleto** (`f` no esta implementado); hay que adaptarlo al problema.

Codigo. Ver `cpp.tex`, seccion *Inclusion-exclusion over subsets*.

2.3 Polinomios y transformadas

2.3.1 FFT (iterativo, complex)

Idea intuitiva. La **Transformada Rapida de Fourier** calcula el DFT de un vector en $O(N \log N)$. La inversa tambien. La convolucion de dos polinomios $A(x) \cdot B(x)$ se obtiene como $\text{IFFT}(\text{FFT}(A) \cdot \text{FFT}(B))$ componente a componente. La version **iterativa** reordena los elementos por bit-reversal antes y aplica “mariposas” (combina pares en $O(1)$).

Propiedades clave (invariantes).

- Trabaja con `complex<double>`; hay errores de redondeo que se acumulan.
- Tamano N debe ser potencia de 2 (se hace `n <= 1` hasta alcanzar `a.size() + b.size()`).
- Resultado se redondea con `llround`.

Complejidad.

- $O(N \log N)$ con N la primera potencia de 2 que cubre el resultado.

Donde tocar para modificar.

- **Modulo arbitrario:** aniadir tres FFTs y “FFT con tres modulos” para reconstruir el resultado con CRT (mejor precision).
- **Tamano mas pequeno:** si sabes que el resultado cabe en $K < a.size() + b.size() - 1$, podes usar $N = 2^{\lceil \log_2 K \rceil}$ para ahorrar.
- **Cambiar precision:** aniadir `long double` y `acosl(-1)` para mayor precision (marginal).

Trampas y errores frecuentes.

- Coeficientes **negativos:** `llround(-0.5)` da 0, no -1; aniadir `floor(x + 0.5)` si hay negativos.
- Overflow al convertir a `long long` si los coeficientes son grandes ($\geq 2^{52}$).

Codigo. Ver `cpp.tex`, seccion *FFT (iterativo, complex)*.

2.3.2 NTT (mod 998244353)

Idea intuitiva. La **Number Theoretic Transform** es la version entera de FFT. En vez de raices de unidad complejas, usa raices modulo un primo p con $p - 1$ divisible por una potencia grande de 2. El primo $998244353 = 119 \cdot 2^{23} + 1$ sirve para NTT hasta $2^{23} = 8 \cdot 10^6$.

Propiedades clave (invariantes).

- **Exacto** modulo p (sin redondeo).
- Raiz primitiva $g = 3$.
- Misma estructura que FFT (bit-reversal + mariposas).

Complejidad.

- $O(N \log N)$.

Donde tocar para modificar.

- **Otro primo:** $985661441 = 235 \cdot 2^{22} + 1$, raiz 3; $754974721 = 45 \cdot 2^{24} + 1$, raiz 11. Ajustar `MOD` y `PRIMITIVE_ROOT`.
- **Multiplicacion de varias cosas:** NTT es lineal; aniadir cache si llamas con los mismos vectores.
- **Poly inv (Newton):** ver el modulo de `polynomial utilities`.

Trampas y errores frecuentes.

- El “exponente magico” $(\text{MOD} - 1) / \text{len}$ requiere que $\text{MOD} - 1$ sea divisible por len ; para 998244353, len debe ser potencia de 2 hasta 2^{23} .

Codigo. Ver `cpp.tex`, seccion *NTT (mod 998244353)*.

2.3.3 Polynomial utilities (mod p)

Idea intuitiva. Capa de utilidades sobre NTT/FFT: suma, resta, **inversa** (Newton iterativo), derivada, integral. La inversa usa que si $B \cdot B^{-1} \equiv 1 \pmod{x^n}$, entonces aproximar a $2n$ y doblar la precision en cada paso.

Propiedades clave (invariantes).

- `polyInv(b, n)`: asume `b[0]` invertible (no cero).
- Usa NTT internamente; misma complejidad $O(N \log N)$.
- `polyInteg` necesita el inverso de $i+1 \pmod p$; se precomputa en una `static vector` que crece a demanda.

Complejidad.

- Operaciones polinomiales $O(N \log N)$.

Donde tocar para modificar.

- **Division**: anadir `polyDiv(a, b)` usando la inversa de b truncada.
- **Multi-point evaluation**: $O(N \log^2 N)$ con divide y venceras.
- **Composition**: muy dificil en general; no recomendado en contests.

Trampas y errores frecuentes.

- `polyInv` asume `b[0]` no es 0 modulo p ; sino, no existe la inversa.
- Olvidar `r.resize(cur)` deja residuos de la iteracion anterior.

Codigo. Ver `cpp.tex`, seccion *Polynomial utilities (mod p)*.

2.4 Algebra lineal

2.4.1 Gauss-Jordan (mod p)

Idea intuitiva. Reduccion gaussiana para resolver sistemas $Ax = b$ y calcular determinante y rango. En la version **Gauss-Jordan**, se elimina **tambien** hacia arriba (no solo triangular inferior). El algoritmo opera sobre la matriz aumentada $[A|b]$ modulo p .

Propiedades clave (invariantes).

- Rango `r`: numero de filas no cero despues de la reduccion.
- **3 casos** para $Ax = b$: sin solucion ($r < n$ y ultima columna no cero en filas nulas), solucion unica ($r = n$), infinitas ($r < n$ y filas nulas tienen $b = 0$).
- Determinante = producto de la diagonal (con signo si se swapean filas).

Complejidad.

- $O(N^3)$ por cubica, $O(N^2)$ para triangularizar.

Donde tocar para modificar.

- **Sin modulo:** anadir `double` y `abs` para determinantes reales.
- **Encontrar la solucion particular con libres en 0:** el snippet ya lo hace; si quieres una **otra** particular, anadir un parametro `freeVars` con valores iniciales.
- **Rango sobre matriz sin aumentar:** pasar solo `A`, no `aug`.

Trampas y errores frecuentes.

- Si el pivote es 0 pero hay un swap posible, **no olvidar** el swap.
- Division por cero: solo se divide por `inv(a[row][col])`, que existe porque `a[row][col]` es no cero y `p` es primo (por eso `modpow(a, p-2, p)`).

Codigo. Ver `cpp.tex`, seccion *Gauss-Jordan (mod p)*.

2.4.2 Matrix exponentiation

Idea intuitiva. Para calcular M^k en $O(N^3 \log k)$, se usa la misma idea que `modpow` pero la multiplicacion es de matrices. Asi, M^k se descompone en bits y se multiplican las matrices correspondientes.

Propiedades clave (invariantes).

- Multiplicacion asociativa: $(M^a)(M^b) = M^{a+b}$.
- Identidad es la matriz diagonal con 1s.
- Si M es la matriz de transicion de un sistema lineal (recurrencia, automata), M^k da el estado tras k pasos.

Complejidad.

- $O(N^3 \log k)$.

Donde tocar para modificar.

- **Aplicar a recurrencia:** para $F_n = aF_{n-1} + bF_{n-2}$, construir $M = \begin{pmatrix} a & b \\ 1 & 0 \end{pmatrix}$, M^n da F_n en `M[0][0] * F_1 + M[0][1] * F_0`.
- **Multiplicacion sin la optimizacion `if (a[i][k])`:** para matrices densas, la guarda no ayuda; quitarla.
- **Matrices pequenas ($N \leq 3$):** hardcodear la multiplicacion puede ser mas rapido.

Trampas y errores frecuentes.

- Multiplicacion `a[i][k] * b[k][j]` puede overflow; el snippet usa `% MOD` al final, pero el producto intermedio puede pasar 2^{63} si los valores son grandes y `MOD` es chico. Mejor hacer el modulo en cada paso.

Codigo. Ver `cpp.tex`, seccion *Matrix exponentiation*.

2.5 Juegos y teoria de juegos

2.5.1 Sprague-Grundy template

Idea intuitiva. Para juegos **imparciales** (ambos jugadores tienen los mismos movimientos), el teorema de Sprague-Grundy dice que cada posicion tiene un numero (el **Grundy** o **mex**) y la posicion es **ganadora** para el jugador actual sii el XOR de los Grundy de los componentes es no cero. El **mex** (“minimum excludant”) es el menor entero no negativo que no esta en el conjunto de Grundy de las posiciones alcanzables.

Propiedades clave (invariantes).

- Posiciones terminales (sin movimientos) tienen Grundy = 0.
- La composición XOR de Grundys da el Grundy del juego disjunto.
- **Caso base:** si `state` no tiene transiciones, Grundy = 0.

Complejidad.

- Depende del juego: $O(\text{estados} \cdot \text{transiciones})$ con memoization.

Donde tocar para modificar.

- **Adaptar al juego concreto:** reemplazar el comentario `// populate 'reachable' ...` con la lógica del juego (ej. para Nim: añadir todos los heap sizes reducidos).
- **Memoization con map:** usar `unordered_map<long long, long long>` si el estado es grande.
- **Computar Grundy de varios estados independientes:** para Nim, es el XOR de los heap sizes; para Kayles, hay tabla precomputada.

Trampas y errores frecuentes.

- Olvidar memoizar: el algoritmo sin cache es exponencial.
- Considerar `unordered_map` vs `map` según el tamaño del espacio de estados.

Código. Ver `cpp.tex`, sección *Sprague-Grundy template*.

2.5.2 Small game tables (Kayles SG values)

Idea intuitiva. Cuando el juego tiene **pocas** posiciones conocidas, conviene **hardcodear** la tabla de Grundys para responder en $O(1)$ por query. Kayles es un juego clásico donde tiras un bolo y tumbas uno o dos adyacentes; sus Grundys para $n \leq 71$ son bien conocidos y están precomputados.

Propiedades clave (invariantes).

- Solo útil para Kayles; otros juegos necesitan su propia tabla.
- La tabla se obtuvo experimentalmente (o por recurrencia) y se almacena literal en el código.

Complejidad.

- $O(1)$ por query.

Donde tocar para modificar.

- **Otro juego:** añadir otra tabla literal (Nim, Dawson's Kayles, Grundy's Game, etc.).
- **Tabla generada por código:** añadir un precompute al inicio que llene la tabla con el template Sprague-Grundy.

Trampas y errores frecuentes.

- Si el problema da n fuera de rango, añadir bounds check antes de indexar.

Código. Ver `cpp.tex`, sección *Small game tables (Kayles SG values)*.

3 Estructuras de datos

3.1 Ordenado y contenedores

3.1.1 Ordered Set (pbds)

Idea intuitiva. La **policy-based data structure** de GCC es un arbol rojo-negro (`rb_tree`) con dos operaciones extra: `find_by_order(k)` devuelve un iterador al k -esimo elemento (0-indexed) en $O(\log n)$, y `order_of_key(x)` cuenta cuantos elementos son **menores** que x en $O(\log n)$. Equivale a tener un **set** con “rango” y “rango por posicion” gratis.

Propiedades clave (invariantes).

- Solo en GCC/clang (header `ext/pb_ds/...`); **no** en MSVC.
- `less_equal<int>` permite multiset; ojo con `erase` que ahí usa `upper_bound`.
- Tamano maximo $\sim 10^7$ elementos (limite de la libreria).

Complejidad.

- $O(\log n)$ por operacion.

Donde tocar para modificar.

- **Multiset:** cambiar `less<int>` por `less_equal<int>` y borrar con `s.erase(s.upper_bound(x))`.
- **Estadísticas de orden:** `find_by_order(k)` da el k -esimo, util para mantener el k -esimo menor dinamicamente.
- **Acceso por rango:** anadir un wrapper que combine `lower_bound` + `order_of_key`.

Trampas y errores frecuentes.

- `find_by_order(n)` (con n fuera de rango) es UB.
- No olvidar `#include <ext/pb_ds/assoc_container.hpp>` y `<ext/pb_ds/trie_policy.hpp>`.

Codigo. Ver `cpp.tex`, seccion *Ordered Set (pbds)*.

3.1.2 Basis (XOR basis)

Idea intuitiva. Una **base de XOR** mantiene un conjunto de mascaras (numeros) tal que cualquier XOR de un subconjunto del input se puede expresar como XOR de un subconjunto de la base. La idea es similar a la base de un espacio vectorial sobre $\mathbb{F}_2$. Construir: insertar cada mascara m recorriendo los bits de mayor a menor; si el bit i de m esta “libre” en la base, se anade; sino, se hace $m \oplus basis[i]$ y se continua.

Propiedades clave (invariantes).

- Tamano maximo de la base: ≤ 64 (bits) o $\leq \log_2(\text{max value})$.
- Base **minimal**: la construccion garantiza que cada vector tiene un unico bit alto distinto.
- El subespacio generado es invariante.

Complejidad.

- Insertar una mascara: $O(\text{bits})$. Construir la base completa: $O(N \cdot \text{bits})$.

Donde tocar para modificar.

- **Obtener el maximo XOR de un subconjunto:** ordenar la base por bit alto descendente y aplicar XOR si aumenta.
- **Obtener el minimo XOR positivo:** el minimo distinto de 0 es el minimo `basis[i]` no cero.
- **Cuantos subconjuntos dan XOR k :** con la base, reducir k y contar grados de libertad.
- **XOR basis en 32 bits:** cambiar `int d = 64` a `int d = 32`.

Trampas y errores frecuentes.

- Si los numeros son `long long`, usar `1LL << i`.
- Si se quiere la base **reducible** (que no necesariamente tenga vectores con unico bit alto), usar otra variante.

Codigo. Ver `cpp.tex`, seccion *Basis (XOR basis)*.

3.1.3 Trie

Idea intuitiva. Un **trie** (o prefix tree) almacena strings donde cada nodo tiene hijos por cada caracter posible (tipicamente 26 letras). Permite insercion, busqueda exacta y busqueda por prefijo en $O(|word|)$.

Propiedades clave (invariantes).

- Cada nodo tiene `links[26]` (punteros a hijos) y un flag `isEnd`.
- Memoria: $O(\text{total chars})$; cada nodo cuesta ~ 26 punteros (puede comprimirse con `map<char, Node*>` si el alfabeto es disperso).
- Las busquedas descienden siguiendo caracteres; si algun enlace es nulo, falla.

Complejidad.

- Insercion y busqueda: $O(|s|)$ donde $|s|$ es el largo del string.

Donde tocar para modificar.

- **Alfabeto mas grande:** cambiar `links[26]` por `links[256]` y eliminar `ch - 'a'`.
- **Trie de bits:** para problemas con mascarar, usar `links[2]` y trabajar con bits en vez de caracteres.
- **Trie persistente:** anadir `vector<Node*> versions;` y clonar nodos en cada insercion (como el persistent segtree).
- **Delete:** implementar recursion que borre nodos sin hijos; cuidado con nodos compartidos por varias palabras.

Trampas y errores frecuentes.

- En este snippet los nodos se crean con `new` y nunca se liberan; en contests esta bien, pero en produccion anadir destructor.

Codigo. Ver `cpp.tex`, seccion *Trie*.

3.2 Union-Find

3.2.1 DSU classico

Idea intuitiva. El **Disjoint Set Union** (Union-Find) mantiene una particion de elementos en conjuntos disjuntos. Las dos operaciones basicas son `find(x)` (devuelve el representante del conjunto de x) y `union(x, y)` (fusiona los conjuntos). La optimizacion **path compression** aplanar el arbol durante `find`, y **union by size/rank** mantiene el arbol balanceado. Combinadas, dan costo **amortizado** $O(\alpha(n))$ por operacion, casi $O(1)$.

Propiedades clave (invariantes).

- `parent[i]` apunta al padre en el arbol; la raiz es `parent[i] == i`.
- `size[i]` solo es valido en la raiz.
- **Representante unico**: cada conjunto tiene exactamente una raiz.

Complejidad.

- $O(\alpha(n))$ amortizado, donde α es la inversa de Ackermann.

Donde tocar para modificar.

- **Aniadir informacion al conjunto** (ej. tamano del conjunto ya esta; suma de los elementos; etc.): aniadir campos al struct y mantenerlos en **union**.
- **DSU para borrar**: borrar el ultimo elemento es trivial, borrar en medio requiere reasignar o rehacer.
- **Weighted DSU**: para resolver ecuaciones tipo $x \equiv a \pmod n$ con restricciones, mantener el “peso” relativo al padre.

Trampas y errores frecuentes.

- `find` recursivo puede overflow del stack para n muy grande; cambiar a iterativo si pasa.
- Olvidar inicializar `size[i] = 1`.

Codigo. Ver `cpp.tex`, seccion *DSU classico*.

3.3 Arboles de segmentos

3.3.1 Segment Tree Add + Sum

Idea intuitiva. Un **segment tree** es un arbol binario donde cada nodo cubre un rango del arreglo. La raiz cubre todo $[0, n)$; cada nodo interno $[l, r]$ tiene dos hijos $[l, m]$ y $[m + 1, r]$. Para mantener la **suma** del rango y soportar **add en rango** eficientemente, se usa **lazy propagation**: cuando un update cubre totalmente un nodo, se aniade el delta a un campo `lazy` y se actualiza el nodo. La propagacion real se hace “bajando” solo cuando es necesario.

Propiedades clave (invariantes).

- `st[node].val`: suma del rango del nodo.
- `st[node].lazy`: pendiente de propagar a los hijos.
- La `push` aplica el `lazy` al nodo actual y propaga a los hijos.

Complejidad.

- Build $O(n)$, update y query $O(\log n)$ amortizado.

Donde tocar para modificar.

- **Cambiar suma por max/min/xor**: solo cambiar la linea `st[node].val = st[node*2].val + st[node*2+1].val`; por la operacion deseada (con identidad apropiada).
- **Cambiar add range por set range**: cambiar el `push` y la operacion en el rango.
- **Aniadir operacion “maximo en rango”**: aniadir nuevo metodo `queryMax(l, r)`.
- **Tamano no potencia de 2**: el snippet fuerza n a potencia de 2 con `while (__builtin_popcount(n) != 1) n++;`; si no quieres padding, aniadir la logica de “nodo vacio”.

Trampas y errores frecuentes.

- En `push`, la condicion `if (l != r)` es crucial: no propagar a hijos si el nodo es hoja (evita UB por `st[node*2]` fuera de rango).

Codigo. Ver `cpp.tex`, seccion *Segment Tree Add + Sum*.

3.3.2 Segment Tree Lazy Assign + Sum

Idea intuitiva. Misma estructura que el anterior pero la operacion es **assign** (setear todos los elementos del rango a un valor), no **add**. Esto **invalida** cualquier lazy add pendiente: si haces **assign** sobre un rango donde ya hay **add** pendiente, el **add** se descarta.

Propiedades clave (invariantes).

- `lazy == LLONG_MAX` indica “sin asignacion pendiente” (valor centinela).
- Al asignar, se reemplaza el valor del nodo por `(r-l+1) * val` y se borran lazies anteriores.

Complejidad.

- Misma que el anterior.

Donde tocar para modificar.

- **Mezclar assign + add:** anadir dos campos `lazy_set` y `lazy_add` y manejar el orden.
- **Operacion no conmutativa:** para maximo o similar, anadir.

Trampas y errores frecuentes.

- El centinela `LLONG_MAX` es valido solo si el problema no usa ese valor en operaciones. Si lo usa, cambiar el centinela.

Codigo. Ver `cpp.tex`, seccion *Segment Tree Lazy Assign + Sum*.

3.3.3 Segment Tree Add + lower bound index

Idea intuitiva. Variante que responde “cual es el primer indice $i \leq \text{bound}$ tal que `val[i] >= v`?”. El truco: el nodo guarda el **maximo** del rango (no la suma). El query desciende priorizando el hijo izquierdo; si su max es $< v$, salta al derecho.

Propiedades clave (invariantes).

- `val = max` del rango (no suma).
- Funciona **solo si** la operacion update es **aditiva** (no assign).

Complejidad.

- $O(\log n)$.

Donde tocar para modificar.

- **Usar suma en vez de max:** anadir dos queries o cambiar el campo.
- **Lower bound con suma, no max:** si quieres “primera posicion donde la suma excede v ”, el truco es el mismo pero manteniendo suma acumulada.
- **Limite superior bound:** necesario para no pasar del rango valido.

Trampas y errores frecuentes.

- Si todos los valores son $< v$, devuelve -1 ; manejar ese caso en el llamador.

Codigo. Ver `cpp.tex`, seccion *Segment Tree Add + lower bound index*.

3.3.4 Segment Tree Lazy Add + Set + Sum

Idea intuitiva. La variante mas completa: combina **add** y **set** en el mismo nodo. Prioridad: el **set** “mata” cualquier **add** previo (porque asignar implica “este es el nuevo valor base”). El **add** se acumula sobre el **set**. El orden importa: **set** primero, **add** despues.

Propiedades clave (invariantes).

- Dos lazies: **lazy_set** (sobrescribe) y **lazy_add** (acumula).
- En **push**: primero aplicar **set** (que borra **add** en hijos), despues **add** (que acumula).

Complejidad.

- $O(\log n)$.

Donde tocar para modificar.

- **Aniadir otro tipo de operacion**: aniadir un nuevo campo y su manejo en **push**.
- **Build desde un arreglo**: ya implementado en el constructor.

Trampas y errores frecuentes.

- Olvidar resetear **lazy_add** cuando se aplica **set** a un hijo.

Codigo. Ver `cpp.tex`, seccion *Segment Tree Lazy Add + Set + Sum*.

3.3.5 Segment Tree 2D

Idea intuitiva. Extender el segment tree a 2 dimensiones: la raiz cubre toda la matriz $[0, n) \times [0, m)$. Cada nodo es un **segment tree 1D** sobre la segunda dimension. Para update en (x, y) , se actualizan $O(\log n)$ nodos externos (uno por nivel vertical) y dentro de cada uno, $O(\log m)$ nodos internos. Query sobre un rectangulo: $O(\log n \cdot \log m)$.

Propiedades clave (invariantes).

- Memoria: $O(n \cdot m)$ en el peor caso (matriz densa).
- Para matrices grandes con valores **cero** y unos pocos updates, se puede comprimir.

Complejidad.

- Update y query $O(\log n \cdot \log m)$, memoria $O(4n \cdot 4m)$.

Donde tocar para modificar.

- **Build desde una matriz**: aniadir constructor que tome `vector<vector<int>>`.
- **Lazy propagation 2D**: aniadir un campo **lazy** por nodo.
- **Suma + maximo simultaneos**: aniadir mas campos.

Trampas y errores frecuentes.

- La recursion puede ser profunda; aniadir iterativo si $n, m > 10^5$.

Codigo. Ver `cpp.tex`, seccion *Segment Tree 2D*.

3.3.6 Segment Tree persistente

Idea intuitiva. Cada `update` crea una **nueva raiz** sin modificar los nodos no afectados. Asi puedes tener **multiples versiones** del arreglo y consultar cada una en $O(\log n)$. Memoria: $O(N \log N)$ nodos en total para N updates.

Propiedades clave (invariantes).

- Cada nodo es **immutable** despues de creado; las versiones comparten estructura.
- `pUpdate` retorna la nueva raiz; la anterior sigue accesible.
- `pQuery` toma una raiz especifica (no la global).

Complejidad.

- Update $O(\log n)$ nodos nuevos, query $O(\log n)$.

Donde tocar para modificar.

- **Persistencia completa (cada update crea un nodo):** anadir `version[v]` para acceder a cada version.
- **Cambiar suma por otro operacion:** ajustar `pUpdate` y `pQuery`.
- **Memoria grande:** en contests, el allocator por defecto puede ser lento; usar `vector<PNode>` con `reserve(2*N*logN)` y un `int nextNode = 0` para asignar en $O(1)$.

Trampas y errores frecuentes.

- Olvidar pasar la **vieja raiz** al query, usando la nueva por error.

Codigo. Ver `cpp.tex`, seccion *Segment Tree persistente*.

3.3.7 Segment Tree iterativo

Idea intuitiva. Representacion **sin recursion**: los nodos se guardan en un **vector** de tamano $2n$. Los hijos del nodo i son $2i$ y $2i + 1$. Las hojas ocupan las posiciones $n, n + 1, \dots, 2n - 1$. Update: subir el cambio desde la hoja hasta la raiz. Query: subir dos indices l y r hasta que se “cruzen”.

Propiedades clave (invariantes).

- $O(\log n)$ por operacion, **mas rapido** en practica que el recursivo (mejor uso de cache, no hay overhead de llamadas).
- Build simple: copiar las hojas y combinar de abajo hacia arriba.

Complejidad.

- $O(\log n)$ por update/query, $O(n)$ build.

Donde tocar para modificar.

- **Suma -> max/min/xor:** cambiar la operacion en `set` y `query`.
- **Anadir update en rango con lazy:** implementar version lazy por separado (no trivial en iterativo).
- **Tamano no potencia de 2:** si n no es potencia de 2, anadir padding.

Trampas y errores frecuentes.

- En el query, `r += n+1` (no `r += n`) porque `query(1, r)` es **inclusivo** en ambos extremos.

Codigo. Ver `cpp.tex`, seccion *Segment Tree iterativo*.

3.4 Otras

3.4.1 MO

Idea intuitiva. Algoritmo de Mo “puro” (sin updates): queries offline ordenadas por $(L/\sqrt{n}, R)$ (con R alternando entre ascendente y descendente segun el bloque). Mover L o R entre queries consecutivas es $O(1)$; el total es $O((N + Q)\sqrt{n})$.

Propiedades clave (invariantes).

- `q` guarda (l, r, idx) .
- El sort prioriza bloque de L ; dentro del bloque, alterna R ascendente/descendente para mejorar la locality.
- Tres punteros `l`, `r` que se mueven paso a paso.

Complejidad.

- $O((N + Q)\sqrt{n})$.

Donde tocar para modificar.

- **Hilbert order:** alternativa al sort clasico; da mejor locality y es mas rapido en practica.
- **Aniadir la operacion concreta:** el snippet tiene `// add` y `// delete` como comentarios; reemplazarlos con la logica del problema (contar distintos, suma de frecuencias, etc.).

Trampas y errores frecuentes.

- Los indices en el sort son **0-based** (`q[i].l = l - 1`), consistente con el resto del codigo.

Codigo. Ver `cpp.tex`, seccion *MO*.

3.4.2 DSU con rollback

Idea intuitiva. Variante del DSU donde cada `unite` se puede deshacer en orden LIFO. La idea: en vez de path compression (que es irreversible), solo se usa union by size. Cada `unite` guarda un par `(parent_changed, old_size)` en una pila. `rollback()` desapila hasta el ultimo “snapshot” (marcador) restaurando los valores.

Propiedades clave (invariantes).

- **Sin path compression:** cada `find` es $O(\log n)$ (peor caso del union by size) en el peor caso, pero amortizado $O(\log n)$.
- **Snapshots:** `snapshot()` marca un punto; `rollback()` deshace hasta ahi.
- Cada `unite` exitoso apila **dos** pares (uno por cada cambio).

Complejidad.

- $O(\log n)$ por `unite/find`, $O(1)$ por `rollback` de una operacion.

Donde tocar para modificar.

- **Rollback parcial** (no todo): implementar snapshots parciales guardando el tamaño del historial.
- **Mas informacion por conjunto:** aniadir mas campos al struct y guardarlos en `history`.
- **Persistente:** aniadir versionado completo (clonar el struct en cada cambio).

Trampas y errores frecuentes.

- El snippet guarda dos pares por `unite`; al rollback **desapilar en orden correcto** (no invertir).
- Olvidar el `components--` en `unite` o el `components++` en rollback deja el contador mal.

Codigo. Ver `cpp.tex`, seccion *DSU con rollback*.

3.4.3 Segment Tree 2D

Idea intuitiva. Extender el segment tree a 2 dimensiones: la raiz cubre toda la matriz $[0, n) \times [0, m)$. Cada nodo es un **segment tree 1D** sobre la segunda dimension. Para update en (x, y) , se actualizan $O(\log n)$ nodos externos (uno por nivel vertical) y dentro de cada uno, $O(\log m)$ nodos internos. Query sobre un rectangulo: $O(\log n \cdot \log m)$.

Propiedades clave (invariantes).

- Memoria: $O(n \cdot m)$ en el peor caso (matriz densa).
- Para matrices grandes con valores **cero** y unos pocos updates, se puede comprimir.

Complejidad.

- Update y query $O(\log n \cdot \log m)$, memoria $O(4n \cdot 4m)$.

Donde tocar para modificar.

- **Build desde una matriz:** anadir constructor que tome `vector<vector<int>>>`.
- **Lazy propagation 2D:** anadir un campo `lazy` por nodo.
- **Suma + maximo simultaneos:** anadir mas campos.

Trampas y errores frecuentes.

- La recursion puede ser profunda; anadir iterativo si $n, m > 10^5$.

Codigo. Ver `cpp.tex`, seccion *Segment Tree 2D*.

3.4.4 Segment Tree persistente

Idea intuitiva. Cada update crea una **nueva raiz** sin modificar los nodos no afectados. Asi puedes tener **multiples versiones** del arreglo y consultar cada una en $O(\log n)$. Memoria: $O(N \log N)$ nodos en total para N updates.

Propiedades clave (invariantes).

- Cada nodo es **immutable** despues de creado; las versiones comparten estructura.
- `pUpdate` retorna la nueva raiz; la anterior sigue accesible.
- `pQuery` toma una raiz especifica (no la global).

Complejidad.

- Update $O(\log n)$ nodos nuevos, query $O(\log n)$.

Donde tocar para modificar.

- **Persistencia completa (cada update crea un nodo):** anadir `version[v]` para acceder a cada version.
- **Cambiar suma por otro operacion:** ajustar `pUpdate` y `pQuery`.
- **Memoria grande:** en contests, el allocator por defecto puede ser lento; usar `vector<PNode>` con `reserve(2*N*logN)` y un `int nextNode = 0` para asignar en $O(1)$.

Trampas y errores frecuentes.

- Olvidar pasar la **vieja raiz** al query, usando la nueva por error.

Codigo. Ver `cpp.tex`, seccion *Segment Tree persistente*.

3.4.5 Segment Tree iterativo

Idea intuitiva. Representacion **sin recursion**: los nodos se guardan en un **vector** de tamaño $2n$. Los hijos del nodo i son $2i$ y $2i + 1$. Las hojas ocupan las posiciones $n, n + 1, \dots, 2n - 1$. Update: subir el cambio desde la hoja hasta la raiz. Query: subir dos índices l y r hasta que se “cruzen”.

Propiedades clave (invariantes).

- $O(\log n)$ por operacion, **mas rapido** en practica que el recursivo (mejor uso de cache, no hay overhead de llamadas).
- Build simple: copiar las hojas y combinar de abajo hacia arriba.

Complejidad.

- $O(\log n)$ por update/query, $O(n)$ build.

Donde tocar para modificar.

- **Suma -> max/min/xor**: cambiar la operacion en **set** y **query**.
- **Aniadir update en rango con lazy**: implementar version lazy por separado (no trivial en iterativo).
- **Tamano no potencia de 2**: si n no es potencia de 2, aniadir padding.

Trampas y errores frecuentes.

- En el query, $r += n+1$ (no $r += n$) porque **query**(l , r) es **inclusivo** en ambos extremos.

Codigo. Ver `cpp.tex`, seccion *Segment Tree iterativo*.

3.5 Fenwick / BIT

3.5.1 BIT point update + prefix sum

Idea intuitiva. El **Fenwick Tree** (BIT) almacena sums parciales de manera que **sum**(i) (suma de $[1, i]$) y **add**(i , Δ) cuestan $O(\log n)$. La representacion es ingeniosa: el nodo i cubre el rango $[i - \text{lowbit}(i) + 1, i]$. Asi, **sum**(i) itera $i -= \text{lowbit}(i)$, sumando los nodos que cubren un prefijo creciente, y **add**(i , Δ) itera $i += \text{lowbit}(i)$, propagando el cambio a todos los nodos cuyo rango contiene a i .

Propiedades clave (invariantes).

- Indice **1-based** (no usar $i = 0$).
- $\text{lowbit}(i) = i \& (-i)$.
- Cada nodo cubre exactamente un rango de tamaño $\text{lowbit}(i)$.

Complejidad.

- $O(\log n)$ por operacion. Memoria $O(n)$.

Donde tocar para modificar.

- **Suma -¿ max/min/xor:** cambiar el += por la operacion (con identidad apropiada). **No** funciona para min de forma natural (no es invertible).
- **Tamano dinamico:** anadir un metodo `resize(newSize)` que preserve valores.
- **Tamano grande:** con $n = 10^7$, mejor comprimir coordenadas.

Trampas y errores frecuentes.

- Indices 0 vs 1: el BIT es **1-based**. Si el problema da indices 0-based, anadir +1 en cada llamada.

Codigo. Ver `cpp.tex`, seccion *BIT point update + prefix sum*.

3.5.2 BIT range add + point query

Idea intuitiva. Truco de **diferencias**: para sumar d a $[l, r]$, anadir d en l y $-d$ en $r + 1$. Asi, el valor en i es la suma de los prefijos hasta i , lo que se calcula con un BIT normal.

Propiedades clave (invariantes).

- Internamente es un BIT 5.1 con la convencion de diferencias.
- `rangeAdd(l, r, delta)` cuesta $O(\log n)$ (dos add).
- `pointQuery(i)` cuesta $O(\log n)$.

Complejidad.

- $O(\log n)$ por operacion.

Donde tocar para modificar.

- **Persistencia:** anadir versionado (cada update crea una nueva “copia” del estado; en BIT no es trivial, mejor usar persistent segtree).
- **Coordenada compression:** si n es grande pero los updates son pocos, comprimir.

Trampas y errores frecuentes.

- Si $r + 1 > n$, **no** anadir `-delta` (no hay efecto fuera del rango); el snippet ya lo chequea.

Codigo. Ver `cpp.tex`, seccion *BIT range add + point query*.

3.5.3 BIT range add + range sum

Idea intuitiva. Para queries $\sum_{i=1}^x a_i$ despues de varios range adds, el truco de diferencias no basta (necesitas la suma acumulada de las diferencias). Solucion: dos BITs, t_1 y t_2 . Tras anadir d a $[l, r]$: t_1 recibe $+d$ en l , $-d$ en $r + 1$; t_2 recibe $+d(l - 1)$ en l , $-d \cdot r$ en $r + 1$. Entonces $\text{prefixSum}(x) = t_1.\text{sum}(x) \cdot x - t_2.\text{sum}(x)$.

Propiedades clave (invariantes).

- Verificacion: para $x \geq r$, $\text{prefixSum}(x)$ acumula correctamente todas las contribuciones.
- La formula sale de expandir la suma de diferencias.

Complejidad.

- $O(\log n)$ por operacion (2 add o 2 sum por cada lado).

Donde tocar para modificar.

- **2D range add + range sum**: anadir otra dimension con la misma tecnica (queda 4 BITS).
- **Cambiar mod**: anadir modulo en cada operacion.

Trampas y errores frecuentes.

- Overflow en `delta * (1 - 1)` si los valores son grandes; usar `long long` y `% MOD` al final.

Codigo. Ver `cpp.tex`, seccion *BIT range add + range sum*.

3.6 RMQ y sparse table

3.6.1 Sparse Table

Idea intuitiva. Precomputa respuestas a queries de tamaño 2^k para $k = 0, 1, \dots, \log n$. Así, `st[k][i]` guarda la respuesta sobre $[i, i + 2^k)$. Para query $[l, r]$, se usan dos bloques de tamaño $2^{\lfloor \log_2(r-l+1) \rfloor}$: $[l, l + 2^k)$ y $[r - 2^k + 1, r)$, y se combinan. Esto da $O(1)$ por query **solo si la operacion es idempotente** (min, max, gcd; **no** suma).

Propiedades clave (invariantes).

- `st[0][i] = a[i]`.
- `st[k][i] = op(st[k-1][i], st[k-1][i + 2^(k-1)])`.
- `LOG = floor(log2(n)) + 1`.

Complejidad.

- Build $O(n \log n)$, query $O(1)$.

Donde tocar para modificar.

- **Cambiar min por otra operacion idempotente**: cambiar las dos ocurrencias de `min(...)` por la operacion (max, gcd). Verificar identidad y asociatividad.
- **Suma en rango**: NO sirve con este metodo; usar prefix sums.
- **Sparse table 2D**: anadir otra dimension (queda $O(n^2 \log^2 n)$ memoria).

Trampas y errores frecuentes.

- El query toma $[l, r]$ **inclusivo** (no $[l, r)$).
- El LOG debe estar bien calculado; con $n = 1$, `LOG = 1`.

Codigo. Ver `cpp.tex`, seccion *Sparse Table*.

3.6.2 Disjoint Sparse Table

Idea intuitiva. Variante que tambien da $O(1)$ por query pero **no requiere idempotencia** para algunas operaciones (sirve para suma, xor, etc. en linea con la mayoria de operaciones asociativas). La idea: para cada nivel k , los rangos “se parten en mitades”. `st[k][i]` cubre un rango que **empieza** en i y se extiende a izquierda o derecha hasta encontrar un “punto de division”.

Propiedades clave (invariantes).

- Construccion diferente al Sparse Table clasico: en cada nivel, se extiende desde cada posicion hacia ambos lados hasta encontrar la mitad de nivel.
- query(l, r) usa $k = \log_2(l \sim r)$ y combina `st[k][l]` y `st[k][r]`.

Complejidad.

- Build $O(n \log n)$, query $O(1)$.

Donde tocar para modificar.

- **Operacion no asociativa:** no funciona; el Disjoint Sparse Table asume asociatividad.
- **Memory compression:** con $n \leq 10^6$ alcanza; con mas, comprimir.

Trampas y errores frecuentes.

- La construccion es sutil; verificar con casos pequenos antes de usar.

Codigo. Ver `cpp.tex`, seccion *Disjoint Sparse Table*.

3.7 Trees y DP en arbol

3.7.1 Binary Lifting / LCA

Idea intuitiva. Precomputa para cada nodo v sus ancestros a distancias 2^k (los “saltos”). Asi, para subir d niveles, se descompone d en binario y se aplican los saltos correspondientes. El **LCA** (Lowest Common Ancestor) se obtiene subiendo el nodo mas profundo hasta igualar profundidades y luego subiendo ambos hasta que sus padres coincidan.

Propiedades clave (invariantes).

- `up[k][v]` = ancestro de v a 2^k niveles.
- `LOG = floor(log2(n)) + 1`.
- La recursion de `dfs` llena `up[0][v]` (el padre directo) y por transitividad los demas.

Complejidad.

- Build $O(n \log n)$, query LCA $O(\log n)$.

Donde tocar para modificar.

- **Distancia en el arbol:** combinar con `depth[]` para calcular `depth[u] + depth[v] - 2*depth[lca]`.
- **K-esimo ancestro:** subir k niveles con el mismo truco.
- **Aniadir peso a las aristas:** guardar `dist[2^k][v]` (distancia acumulada a 2^k saltos) ademas de `up`.

Trampas y errores frecuentes.

- El arbol debe ser **rooted** (el snippet asume eso); si no, hacer un `dfs` desde cualquier nodo.
- Para grafos no conexos, correr `dfs` desde cada componente.

Codigo. Ver `cpp.tex`, seccion *Binary Lifting / LCA*.

3.7.2 Heavy-Light Decomposition (HLD)

Idea intuitiva. Descomponer un arbol en **cadena pesadas** (paths donde cada nodo (salvo la raiz) tiene al hijo “pesado” como hijo de tamano maximo de su subarbol). Cada cadena se mapea a un rango contiguo en un arreglo base; asi, queries sobre **paths** se descomponen en $O(\log n)$ queries sobre rangos contiguos (cada cadena es un rango), que se resuelven con un BIT o segment tree.

Propiedades clave (invariantes).

- `heavy[v]` = hijo pesado de v (el de mayor subarbol).
- `head[v]` = cabeza de la cadena que contiene a v .
- `pos[v]` = posicion en el arreglo base.

Complejidad.

- Build $O(n)$, query sobre path $O(\log^2 n)$ (con segtree) o $O(\log n)$ (con BIT si solo se hace suma).

Donde tocar para modificar.

- **Usar con segment tree:** anadir un segtree sobre el arreglo base; `pathSegments` da los rangos a actualizar/query.
- **Cambiar operacion:** ajustar el segtree (suma, max, etc.).
- **Subtree query:** `subtree[v]` es el rango `[pos[v], pos[v] + size[v] - 1]` en el arreglo base.

Trampas y errores frecuentes.

- `pathSegments` requiere un `segtree` o BIT externo para ser util; por si solo solo da los rangos.

Codigo. Ver `cpp.tex`, seccion *Heavy-Light Decomposition (HLD)*.

3.7.3 Centroid Decomposition

Idea intuitiva. Descomponer recursivamente el arbol en **centroides**. Un centroide de un arbol es un nodo tal que cada subtree (al removerlo) tiene tamano $\leq n/2$. Existe siempre y se encuentra en $O(n)$. Esto da una estructura de **arbol de centroides** de profundidad $O(\log n)$, util para problemas “para cada nodo, encontrar el mas cercano que cumple X”.

Propiedades clave (invariantes).

- En cada nivel, los subarboles se procesan recursivamente.
- `removed[]` marca nodos ya usados como centroide.
- `centroidParent[]` da el padre en el arbol de centroides.

Complejidad.

- Build $O(n \log n)$, queries dependen del problema.

Donde tocar para modificar.

- **Query tipica:** en cada nivel, “expandir” desde el centroide contando caminos validos y usar una estructura auxiliar (set, multiset).
- **Distancia maxima a un nodo “malo”:** mantener un multiset de distancias en cada centroide.
- **Cambiar a otro tipo de descomposicion:** si el problema se adapta mejor a small-to-large o DSU on tree, usar esos.

Trampas y errores frecuentes.

- El snippet actual **solo construye** la descomposicion; las queries se anaden segun el problema.
- Olvidar resetear `removed[]` causa recursion infinita.

Codigo. Ver `cpp.tex`, seccion *Centroid Decomposition*.

3.7.4 Euler Tour + range queries

Idea intuitiva. Un **Euler Tour** (first-time visit) numera los nodos en el orden en que se visitan. El subarbol de v corresponde a un **rango contiguo** $[tin[v], tout[v]]$ en este orden. Así, queries sobre subarboles se resuelven con un BIT o segment tree sobre el arreglo de tiempos.

Propiedades clave (invariantes).

- $tin[v]$ = tiempo de primera visita a v .
- $tout[v]$ = tiempo al **terminar** de procesar el subarbol de v .
- El subarbol de v ocupa las posiciones $[tin[v], tout[v]]$.

Complejidad.

- Build $O(n)$, query sobre subarbol $O(\log n)$ con BIT/segtree.

Donde tocar para modificar.

- **Aniadir valor por nodo:** guardar el valor en $tin[v]$ y operar sobre el rango.
- **Path query:** el Euler Tour **no** sirve para paths; usar HLD o LCA.
- **Multiple queries offline:** procesar en orden de tiempo y mantener una estructura por profundidad.

Trampas y errores frecuentes.

- El snippet solo construye el tour; las queries concretas se aniaden.

Codigo. Ver `cpp.tex`, seccion *Euler Tour + range queries*.

3.8 BST balanceado

3.8.1 Treap implicito

Idea intuitiva. Un **treap** es un arbol binario de busqueda donde cada nodo tiene una **clave** (el valor de la secuencia) y una **prioridad** aleatoria. La invariante es: BST por clave, heap por prioridad. Esto da un arbol **esperado balanceado** sin necesidad de rebalanceo explicito. Un **treap implicito** no almacena una clave; la posicion de un nodo en la secuencia esta dada por el **tamano del subarbol izquierdo**, permitiendo split/merge por posicion.

Propiedades clave (invariantes).

- $sz(node)$ = tamano del subarbol (1 + izquierdo + derecho).
- $split(t, k, l, r)$: parte t en l (primeros k nodos) y r (resto).
- $merge(l, r)$: une dos treaps (todos los de l antes que los de r).
- La prioridad aleatoria asegura equilibrio esperado.

Complejidad.

- $O(\log n)$ **esperado** por operacion (split, merge, insert, erase).

Donde tocar para modificar.

- **Treap con clave (no implicito):** usar `key` en vez de posicion; cambiar `split` por `splitByKey`.
- **Lazy propagation:** aniadir campos `lazy` y aplicarlos en `push` (recursivo).
- **Persistencia:** aniadir versionado clonando nodos.
- **Mejor aleatoriedad:** en lugar de `rand()`, usar `rng()` (mejor estado).

Trampas y errores frecuentes.

- En el snippet actual, `Node(int v)` usa `rand()` que en algunos compiladores da maxima prioridad 0; anadir `rng()` para mejor distribucion.

Codigo. Ver `cpp.tex`, seccion *Treap implicito*.

3.8.2 Mo con updates

Idea intuitiva. Extension del algoritmo de Mo para queries que mezclan **rango** y **updates**. Las queries se ordenan por $(L/\text{blockSize}, R/\text{blockSize}, T)$ donde T es el “tiempo” (numero de updates). Asi, moverse entre queries consecutivas cuesta $O(n^{2/3})$ amortizado.

Propiedades clave (invariantes).

- Tres punteros: L, R, T .
- `applyUpdate` y `revertUpdate` son **inversos** exactos.
- `addPos` y `removePos` mantienen el estado.

Complejidad.

- $O((N + Q) \cdot n^{2/3})$ donde $n^{2/3}$ es el tamano de bloque optimo.

Donde tocar para modificar.

- **El corazon es el “add/remove”:** anadir la logica del problema (ej. contar colores distintos, suma de frecuencias, etc.).
- **Cambiar blockSize:** si la mayoria de updates vs queries cambia, ajustar.
- **Rollback:** anadir `save()` / `restore()` para volver a estados anteriores.

Trampas y errores frecuentes.

- El snippet actual es solo el **esqueleto**; `addPos/removePos/applyUpdate/revertUpdate` deben ser implementados por problema.
- Olvidar `revertUpdate` es el error mas comun.

Codigo. Ver `cpp.tex`, seccion *Mo con updates*.

3.8.3 sqrt-decomposition generico

Idea intuitiva. Dividir el arreglo en **bloques** de tamano $B \approx \sqrt{n}$. Mantener informacion precomputada por bloque (suma, max, etc.). Queries de rango se resuelven combinando bloques enteros ($O(\sqrt{n})$ en total) y elementos sueltos en los bloques parciales. Updates puntuales actualizan el valor y el resumen del bloque.

Propiedades clave (invariantes).

- $B = \lfloor \sqrt{n} \rfloor + 1$.
- `bucket[i]` = resumen del bloque i -esimo (suma, max, etc.).
- `rangeSum(l, r)`: 3 casos segun l y r esten en el mismo bloque o no.

Complejidad.

- $O(\sqrt{n})$ por operacion.

Donde tocar para modificar.

- **Cambiar suma por max/min:** ajustar `bucket` y la combinacion.
- **Aniadir update en rango:** si la operacion es asociativa, sumar al resumen del bloque + ajustar elementos sueltos.
- **Tamano de bloque optimo:** si las queries son muy largas, $B = n^{2/3}$ (sirve para Mo con updates).

Trampas y errores frecuentes.

- Si n no es multiplo de B , el ultimo bloque es mas chico; aniadir check.

Codigo. Ver `cpp.tex`, seccion *sqrt-decomposition generico*.

4 Strings

4.1 Hashing

4.1.1 Rolling Hash (doble modulo)

Idea intuitiva. Un **rolling hash** asigna a cada prefijo del string un valor numerico tal que el hash de un substring se puede obtener en $O(1)$ a partir de los prefijos. La forma clasica: $h[i] = h[i-1] \cdot A + s[i-1]$ (donde A es una base y todo modulo M). Asi, $h[r] - h[l-1] \cdot A^{r-l+1}$ da el hash del substring $[l, r]$ (con A^{r-l+1} precomputado). Usar **dos modulos** distintos reduce drasticamente la probabilidad de colision.

Propiedades clave (invariantes).

- $h[0] = 0$, $h[i]$ = hash de los primeros i caracteres.
- $p[i] = A^i \bmod M$ precomputado.
- Comparar dos substrings: comparar sus pares de hashes.
- **Colisiones:** posibles (probabilidad $1/M$ por par). Doble hash: $\approx 1/(M_1 M_2)$.

Complejidad.

- Precompute $O(n)$, query hash $O(1)$.

Donde tocar para modificar.

- **Caracteres con valor > 26:** si los strings incluyen simbolos, aniadir `if (c >= 'a' && c <= 'z') v = c - 'a'; else v = 26 + ....`
- **Hash de substrings con updates:** aniadir un Fenwick tree / segment tree sobre los hashes (no es trivial).
- **Cambiar mod:** usar dos primos grandes ($10^9 + 7$ y $10^9 + 9$) o un `u128` para un solo hash sin mod.
- **Base aleatoria:** aniadir random para evitar ataques (en contests no es necesario; en produccion si).

Trampas y errores frecuentes.

- Si los substrings a comparar estan vacios ($l > r$), su hash es 0; aniadir check.
- **Overflow** en $h[i-1] * A$: usar `long long` y aplicar `% M` en cada operacion (no al final).

Codigo. Ver `cpp.tex`, seccion *Rolling Hash (doble modulo)*.

4.2 Patrones

4.2.1 KMP + LPS

Idea intuitiva. El algoritmo **Knuth-Morris-Pratt** busca todas las ocurrencias de un patron P en un texto T en $O(|T| + |P|)$. La clave es la tabla **LPS** (Longest Proper Prefix which is Suffix) que, para cada posicion i del patron, dice el largo del prefijo mas largo que tambien es sufijo **propio**. Durante el match, cuando hay mismatch, en vez de volver al inicio, saltamos al LPS anterior, evitando trabajo redundante.

Propiedades clave (invariantes).

- $lps[i]$ = largo del prefijo mas largo que es sufijo propio de $p[0..i]$.
- El match usa dos punteros i (texto) y j (patron); en mismatch, $j = lps[j - 1]$.
- Cuando $j == |P|$, se encontro una ocurrencia; continuar con $j = lps[j - 1]$ para buscar mas.

Complejidad.

- $O(|T| + |P|)$.

Donde tocar para modificar.

- **Contar ocurrencias:** el snippet ya cuenta; para devolver posiciones, anadir un `vector<int>`.
- **Varios patrones:** usar Aho-Corasick en su lugar.
- **Matcher exacto con wildcard:** anadir logica para que `?` matchee cualquier caracter.
- **Tamano pequeno:** si el patron es ≤ 5 caracteres, Rabin-Karp es mas simple.

Trampas y errores frecuentes.

- En el snippet, i y j son 0-based. La condicion `j == sz_pattern` para contar funciona.

Codigo. Ver `cpp.tex`, seccion *KMP + LPS*.

4.2.2 Z-algorithm

Idea intuitiva. La **Z-function** $Z[i]$ de un string S es el largo del prefijo mas largo de S que **tambien** empieza en $S[i]$. Asi, $Z[0] = n$ por convencion. Se calcula en $O(n)$ manteniendo una ventana $[l, r]$ que es el match mas largo encontrado hasta ahora. La consulta $S[l..r] = S[0..r - l]$ da informacion sobre $Z[i]$ cuando $i \leq r$.

Propiedades clave (invariantes).

- Construir Z cuesta $O(n)$ con el truco de la ventana.
- Para matching de patron P en texto T : concatenar $P + \# + T$ y anadir todos los i con $Z[i] = |P|$.

Complejidad.

- $O(n)$ para construir; matching $O(n + m)$.

Donde tocar para modificar.

- **Detectar bordes (border):** $Z[i] = n - i$ indica que $S[0..n - i - 1]$ es borde.
- **Prefijo comun mas largo de dos strings:** anadir $S_1 + \# + S_2$; $Z[|S_1| + 1]$ es el prefijo comun.
- **Comparar prefijos y sufijos:** combinar $S + \# + \text{reverse}(S)$.

Trampas y errores frecuentes.

- $Z[0]$ siempre vale n (no se calcula).
- Si $i > r$, la ventana no aporta; empezar desde 0.

Codigo. Ver `cpp.tex`, seccion *Z-algorithm*.

4.2.3 Rabin-Karp (single hash)

Idea intuitiva. Rolling hash aplicado a pattern matching: calcular el hash del patron P y de cada ventana de tamano $|P|$ en T . Si los hashes coinciden, verificar caracter por caracter (para descartar colisiones). Esto da $O(n + m)$ **esperado**.

Propiedades clave (invariantes).

- Si los hashes difieren, los substrings son distintos (sin colision).
- Si coinciden, **verificar** (la verificación es $O(m)$ pero rara vez se ejecuta).
- Peor caso (muchas colisiones): $O(nm)$.

Complejidad.

- Esperado $O(n + m)$, peor $O(nm)$.

Donde tocar para modificar.

- **Doble hash**: añadir un segundo par (BASE2, MOD2) para reducir colisiones.
- **Varios patrones**: añadir un set de hashes y buscar match con cualquiera.
- **Tamaño de ventana variable**: si los patrones tienen largos distintos, usar el largo específico por patron.

Trampas y errores frecuentes.

- Overflow en $(tHash - text[i] * h) * BASE$: aplicar mod correctamente y añadir +MOD antes de multiplicar.

Código. Ver `cpp.tex`, sección *Rabin-Karp (single hash)*.

4.2.4 Aho-Corasick (basic)

Idea intuitiva. Construye un **trie** con todos los patrones y añade **failure links** analogos a los de KMP: el failure link de un nodo v apunta al nodo mas largo que es sufijo del string representado por v y sigue siendo prefijo de algun patron. Así, al recorrer el texto, si hay mismatch, saltamos al failure link. Esto permite buscar **todos** los patrones en $O(|T| + \sum |P_i|)$.

Propiedades clave (invariantes).

- El trie almacena los patrones.
- `link[v]` (failure link) se calcula en BFS en orden de profundidad creciente.
- `out[v]` lista los IDs de patrones que terminan en v .
- Para queries eficientes, añadir **output links**: propagar out de los failure links a los hijos.

Complejidad.

- Build $O(\sum |P_i|)$, query $O(|T| + \text{ocurrencias})$.

Donde tocar para modificar.

- **Output links en build**: ya hecho en el snippet `(for(int pid : t[t[u].link].out) t[u].out.push_back(pid);)`.
- **Transiciones automaticas**: añadir el goto `t[v].next[k] = t[t[v].link].next[k]` cuando el enlace directo es -1; así el bucle de matching es un solo paso por caracter.
- **Aplicar a problemas tipo DP**: contar formas de construir strings evitando los patrones (ver el siguiente modulo).

Trampas y errores frecuentes.

- La BFS debe empezar con la raíz en la cola y los hijos de la raíz con `link = 0`.
- Olvidar propagar out da ocurrencias perdidas.

Código. Ver `cpp.tex`, sección *Aho-Corasick (basic)*.

4.2.5 Aho-Corasick DP for forbidden patterns

Idea intuitiva. Contar strings de longitud L que **evitan** un conjunto de patrones. Se modela como una DP sobre el **estado del automaton** de Aho-Corasick: $dp[v]$ = numero de formas de llegar al estado v despues de cierto numero de caracteres. Transicion: por cada caracter c en el alfabeto, ir al estado $next[v][c]$, pero **solo** si ese estado no tiene un patron (de lo contrario, el string contendria el patron prohibido).

Propiedades clave (invariantes).

- dp se mantiene por nivel (longitud del string construido).
- Transicion: $ndp[u] = \sum_c dp[v]$ donde $u = next[v][c]$ y u no es “terminal” (no tiene patron que termina ahi).
- Respuesta: suma de $dp[v]$ sobre todos los estados v no terminales al final.

Complejidad.

- $O(L \cdot |S| \cdot |\Sigma|)$ donde $|S|$ es el numero de estados y $|\Sigma|$ el alfabeto.

Donde tocar para modificar.

- **Alfabeto mas pequeno:** si el problema es solo $\{0, 1\}$, $|\Sigma| = 2$.
- **Permitir ocurrencias parciales:** cambiar la condicion a “no estado terminal” o relajar segun el problema.
- **Recuperar el string:** anadir backtracking si el problema lo pide.

Trampas y errores frecuentes.

- El snippet asume Aho-Corasick ya construido con transiciones automaticas. Sin eso, anadir el goto en el bucle.

Codigo. Ver `cpp.tex`, seccion *Aho-Corasick DP for forbidden patterns*.

4.3 Estructuras avanzadas

4.3.1 Suffix Array

Idea intuitiva. Un **suffix array** SA es un arreglo que contiene las posiciones de inicio de los sufijos del string S , ordenados lexicograficamente. Se construye en $O(n \log n)$ con doubling: ordenar por el primer caracter, luego por los primeros 2, luego 4, etc. Para hacer cada paso lineal, se usa **counting sort** sobre los ranks.

Propiedades clave (invariantes).

- $p[i]$ = posicion de inicio del i -esimo sufijo (ordenado).
- $c[i]$ = rank del sufijo que empieza en i .
- Cada iteracion duplica la longitud de la “ventana” considerada.
- El algoritmo termina cuando $2^k \geq n$.

Complejidad.

- $O(n \log n)$ con counting sort.

Donde tocar para modificar.

- **Suffix array en $O(n)$:** usar SA-IS (mas complejo, no incluido).
- **Buscar patron:** con SA + LCP es $O(|P| \log n)$ por busqueda binaria.
- **Suffix array de varios strings:** anadir centinelas (\$) entre ellos.

Trampas y errores frecuentes.

- La “ventana” 2^k se hace circular: $p[i] = (p[i] - (1 \ll k) + n) \% n$.
- Para contar bien los ranks duplicados, comparar pares $(c[i], c[(i+2^k) \% n])$.

Codigo. Ver `cpp.tex`, seccion *Suffix Array*.

4.3.2 LCP Array (Kasai)

Idea intuitiva. El **LCP array** (Longest Common Prefix) guarda, para cada par de sufijos adyacentes en el SA, el largo de su prefijo comun. El algoritmo de **Kasai** lo calcula en $O(n)$ iterando los sufijos **en orden del string original** (no del SA), y reutiliza el LCP previo.

Propiedades clave (invariantes).

- $lcp[i] = \text{LCP entre los sufijos en } sa[i] \text{ y } sa[i-1]$.
- Se necesita `rank_[i]` (la inversa del SA) para iterar.
- El truco de Kasai: si $lcp > 0$ para el sufijo anterior, el actual parte de al menos $lcp - 1$.

Complejidad.

- $O(n)$.

Donde tocar para modificar.

- **RMQ sobre LCP:** para queries “LCP de dos sufijos”, encontrar sus ranks y hacer RMQ en `lcp`.
- **Contar substrings distintas:** $\sum (n - sa[i] - lcp[i])$.
- **Longest common substring:** maximo en `lcp`.

Trampas y errores frecuentes.

- Si dos sufijos son iguales (raro, solo si el string tiene $\text{period} < n$), `lcp` puede ser $n - sa[i]$.

Codigo. Ver `cpp.tex`, seccion *LCP Array (Kasai)*.

4.3.3 Suffix Automaton

Idea intuitiva. Un **suffix automaton** es un DFA minimo que reconoce todos los substrings del string original. Tiene la propiedad magica de tener $\leq 2n - 1$ estados. Cada estado representa una clase de equivalencia de substrings (los que tienen el mismo conjunto de posiciones finales). El **link** de un estado apunta al estado del sufijo propio mas largo.

Propiedades clave (invariantes).

- `len[v]`: largo maximo de los substrings de la clase.
- `link[v]`: padre en el arbol de sufijos (suffix link).
- `next[v][c]`: transicion por caracter.
- `occ[v]`: numero de posiciones finales (cuantas veces aparece el patron).

Complejidad.

- Build $O(n)$ con la operacion de **clonar** estados.

Donde tocar para modificar.

- **Contar substrings distintas:** $\sum_{v \neq 0} (\text{len}[v] - \text{len}[\text{link}[v]])$.
- **Longest common substring:** navegar el SAM con el segundo string, manteniendo el match actual.
- **Substring count:** despues del build, ordenar estados por **len** descendente y propagar **occ**.
- **Endpos queries:** el SAM es el DFA canonico de los substrings; queries “cuantas veces aparece P ” se responden en $O(|P|)$.

Trampas y errores frecuentes.

- El **clone** debe tener **occ** = 0 (no es una nueva posicion final); el **link** del estado siendo creado y del estado clonado deben apuntar al clon.
- Saltarse esto da SAM incorrecto.

Codigo. Ver `cpp.tex`, seccion *Suffix Automaton*.

4.3.4 Lyndon factorization (Duval)

Idea intuitiva. Un **string de Lyndon** es un string que es estrictamente menor que todos sus rotaciones. Toda cadena tiene una **factorizacion unica** de Lyndon (algoritmo de **Duval**): una secuencia de strings de Lyndon $L_1, L_2, \dots, L_k$ tales que $L_1 \geq L_2 \geq \dots \geq L_k$ y la concatenacion es el string original. Esto da $O(n)$ para encontrar los cortes.

Propiedades clave (invariantes).

- Cada factor de Lyndon es de la forma uv donde u se repite y v es prefijo de u .
- La longitud del factor esta dada por la “longitud del periodo”.
- Aplicacion: el “Lyndon word” es el menor en orden lexicografico entre todas las rotaciones.

Complejidad.

- $O(n)$.

Donde tocar para modificar.

- **Encontrar la menor rotacion:** el menor sufijo lexicografico de $s + s$ es la menor rotacion; la posicion se obtiene con un solo pase de Duval.
- **Algoritmo de Booth:** alternativa para rotacion minima, tambien $O(n)$.
- **Validar:** si necesitas verificar que un string es de Lyndon, compararlo con sus rotaciones.

Trampas y errores frecuentes.

- La condicion $s[j] \leq s[k]$ (no $<$) en el while de Duval es crucial; cambiar a $<$ da comportamiento distinto.

Codigo. Ver `cpp.tex`, seccion *Lyndon factorization (Duval)*.

4.4 Palindromos

4.4.1 Manacher

Idea intuitiva. Calcula en $O(n)$ los **radios** de todos los palindromos centrados en cada posicion (impares y pares). La idea: mantener una ventana $[l, r]$ que es el palindromo centrado en un punto anterior mas largo encontrado. Si el nuevo centro i cae dentro de $[l, r]$, entonces el palindromo centrado en i tiene al menos el radio del palindromo “espejo” $l + r - i$; sino, empezar desde 1 (impar) o 0 (par). Despues, expandir lo que falte.

Propiedades clave (invariantes).

- Conveccion del snippet: $\text{odd}[i] = k$ tal que el palindromo impar mas largo centrado en i tiene longitud $2k - 1$. $\text{even}[i] = k$ para par centrado entre $i - 1$ e i , longitud $2k$.
- k cuenta tambien cuantos palindromos hay centrados ahi.
- Las queries $\text{isPalindrome}(l, r)$ son $O(1)$.

Complejidad.

- Construccion $O(n)$, queries $O(1)$.

Donde tocar para modificar.

- **Solo contar palindromos:** ya implementado en `countPalindromes`.
- **Longest palindromic substring:** ya en `longestPalindrome`.
- **Modificar la conveccion de radios:** si quieres “radio = mitad de la longitud” en vez de “k palindromos”, ajustar el offset.
- **Queries con k no precomputado:** anadir helpers especificos.

Trampas y errores frecuentes.

- La conveccion del snippet es **distinta** de la “clasica de radio” (donde $\text{radio} = (L - 1)/2$); tener cuidado al copiar a otro codigo.

Codigo. Ver `cpp.tex`, seccion *Manacher*.

5 Grafos

5.1 Caminos minimos

5.1.1 Dijkstra

Idea intuitiva. Encuentra el shortest path desde un origen s a todos los vertices en $O((V + E) \log V)$ asumiendo **pesos no negativos**. La idea: usar una cola de prioridad (min-heap) ordenada por distancia tentativa. En cada paso, extraer el nodo con menor distancia; si ya fue procesado, saltar. Si al actualizar un vecino la distancia mejora, insertar/actualizar en la cola.

Propiedades clave (invariantes).

- Asume pesos ≥ 0 ; con pesos negativos usar Bellman-Ford.
- Greedy correctness: la primera vez que se extrae un nodo, su distancia es la definitiva.
- Si se quiere reconstruir el camino, guardar el `parent[]`.

Complejidad.

- $O((V + E) \log V)$ con heap binario, $O(V + E)$ con heap de Fibonacci.

Donde tocar para modificar.

- **Camino mas corto entre todos los pares:** si se necesita desde todos los origenes, usar Floyd o ejecutar Dijkstra V veces.
- **Shortest path con aristas negativas limitadas:** Bellman-Ford.
- **Reconstruir camino:** anadir `parent[v]` actualizado junto a `dist[v]`.
- **Dijkstra con potentials (Johnson):** para aristas negativas si se puede reescalar.

Trampas y errores frecuentes.

- Distancias no inicializadas a INF; el snippet usa `1e18`.
- El `if(dist[adjN] != 1e18)` para borrar de la cola es importante si usas `set` (no así con `priority_queue`).

Codigo. Ver `cpp.tex`, seccion *Dijkstra*.

5.1.2 Bellman-Ford

Idea intuitiva. Encuentra shortest path desde s incluso con **pesos negativos** en $O(VE)$. La idea: relajar todas las aristas $V - 1$ veces. Cada relajacion propaga la mejor distancia a “un paso mas”. Despues de $V - 1$ pasadas, si alguna arista aun se puede relajar, hay un **ciclo negativo** alcanzable.

Propiedades clave (invariantes).

- Funciona con pesos negativos (sin ciclos negativos).
- Tras $V - 1$ relajaciones, las distancias son definitivas (si no hay ciclo negativo).
- Una relajacion adicional detecta ciclos negativos: si `dist[u] + w < dist[v]`, hay ciclo negativo alcanzable desde s .

Complejidad.

- $O(VE)$.

Donde tocar para modificar.

- **SPFA**: optimizacion que relaja solo nodos cuya distancia cambio; en el peor caso sigue siendo $O(VE)$.
- **Detectar ciclo negativo global**: ejecutar Bellman-Ford desde un super-origen conectado a todos.
- **Camino mas corto con K aristas exactas**: K iteraciones de Bellman-Ford.

Trampas y errores frecuentes.

- Overflow si los pesos son $\geq 2^{52}$; usar `long long` y `__int128` si hace falta.

Codigo. Ver `cpp.tex`, seccion *Bellman-Ford*.

5.1.3 Floyd-Warshall

Idea intuitiva. Shortest path entre **todos los pares** de vertices en $O(V^3)$. DP: $d[i][j]$ = shortest path usando solo nodos intermedios en $\{0, \dots, k\}$. Transicion: $d_{k+1}[i][j] = \min(d_k[i][j], d_k[i][k+1] + d_k[k+1][j])$.

Propiedades clave (invariantes).

- Detecta ciclos negativos: tras el algoritmo, $d[i][i] < 0$ indica ciclo negativo alcanzable desde i .
- Funciona con pesos negativos (sin ciclos negativos).
- Memoria $O(V^2)$; con $V \leq 400$ es OK, con mas conviene Dijkstra V veces.

Complejidad.

- $O(V^3)$ tiempo, $O(V^2)$ memoria.

Donde tocar para modificar.

- **Reconstruir camino**: anadir `next[i][j]` y backtrack.
- **Transitividad**: el algoritmo tambien calcula transitividad (`d[i][j] != INF` significa alcanzable).
- **Pesos doubles**: anadir tolerancia (`d[i][j] > d[i][k] + d[k][j] - eps`).

Trampas y errores frecuentes.

- Para detectar ciclo negativo correctamente, añadir un chequeo `if (ans[i][i] < 0)` tras el triple bucle.

Codigo. Ver `cpp.tex`, seccion *Floyd-Warshall*.

5.2 Componentes y cortes

5.2.1 Kosaraju SCC

Idea intuitiva. Encuentra las **componentes fuertemente conexas** (SCCs) en $O(V + E)$. Algoritmo en 3 pasos:

- DFS en G rellenando una pila en **post-orden** (los nodos que terminan tarde quedan arriba).
- Construir G^T (grafo transpuesto).
- DFS en G^T procesando nodos en orden de la pila (de mayor a menor tiempo); cada DFS da una SCC.

Propiedades clave (invariantes).

- Cada SCC es maximal fuertemente conexa.
- El **grafo de SCCs** (metagraf) es un DAG.
- Kosaraju y Tarjan dan el mismo resultado; Tarjan es “mas rapido” en practica (un solo DFS).

Complejidad.

- $O(V + E)$.

Donde tocar para modificar.

- **2-SAT**: SCC sobre grafo de implicaciones.
- **Construir el DAG de SCCs**: añadir aristas entre SCCs diferentes.
- **Tamano de cada SCC**: añadir contador durante el segundo DFS.

Trampas y errores frecuentes.

- El snippet usa `vector<...> adj[n]` (VLA); reemplazable por `vector<vector<pair<int,int>>` para portabilidad.

Codigo. Ver `cpp.tex`, seccion *Kosaraju SCC*.

5.2.2 Tarjan (puentes)

Idea intuitiva. Encuentra los **puentes** (bridges): aristas cuya remocion **desconecta** el grafo. Usa un solo DFS manteniendo `tin[v]` (tiempo de descubrimiento) y `low[v]` (el menor `tin` alcanzable desde v via back-edges). Una arista $v - u$ es puente si `low[u] > tin[v]`.

Propiedades clave (invariantes).

- `low[v] = min(tin[v], tin[w])` para todo back-edge (v, w) , y `min(low[u])` para hijos en el DFS.
- Solo se consideran las aristas del **arbol DFS** (no back-edges) para `low`.

Complejidad.

- $O(V + E)$.

Donde tocar para modificar.

- **Bridge tree:** contractar cada 2-edge-connected component a un nodo; el resultado es un arbol.
- **Articulation points:** variante con condicion diferente (ver el siguiente modulo).
- **Grafo no dirigido:** solo funciona en no dirigidos; en dirigidos hay conceptos analogos (strong bridges, dominator tree).

Trampas y errores frecuentes.

- No confundir el parent con un back-edge al padre; chequear `if (adjN == parent) continue;` y contar solo las veces que se “visita” como hijo.

Codigo. Ver `cpp.tex`, seccion *Tarjan (puentes)*.

5.2.3 Tarjan (puntos de articulacion)

Idea intuitiva. Encuentra los **puntos de articulacion** (cut vertices): vertices cuya remocion **desconecta** el grafo. Mismo setup que para bridges: `tin[v]`, `low[v]`. Un nodo v (no raiz) es de articulacion si existe un hijo u con $low[u] \geq tin[v]$. La **raiz** es de articulacion si tiene ≥ 2 hijos en el DFS tree.

Propiedades clave (invariantes).

- $low[u] \geq tin[v]$ significa que el subarbol de u no tiene back-edge “por encima” de v .
- La raiz es caso especial: tiene que perder mas de un hijo para que el grafo se parta.

Complejidad.

- $O(V + E)$.

Donde tocar para modificar.

- **Block-cut tree:** estructura bipartita con BCCs (2-vertex-connected components) y articulation points.
- **Verificar biconectividad:** chequear que no hay puntos de articulacion.

Trampas y errores frecuentes.

- No olvidar el caso especial de la raiz.

Codigo. Ver `cpp.tex`, seccion *Tarjan (puntos de articulacion)*.

5.2.4 Tarjan SCC

Idea intuitiva. Variante del SCC que usa **un solo DFS** con una pila explicita. Mantiene `disc[v]`, `low[v]`, y una pila de nodos “en la SCC actual”. Cuando `low[v] == disc[v]`, v es la raiz de una SCC; popear hasta v inclusive.

Propiedades clave (invariantes).

- Mas eficiente que Kosaraju (un solo DFS).
- $low[v] = \min(low[u], disc[w])$ para back-edges o recursion.
- La pila asegura que solo los nodos en la SCC actual se popean.

Complejidad.

- $O(V + E)$.

Donde tocar para modificar.

- **Tamano de cada SCC:** anadir contador durante el pop.
- **Grafo condensation:** anadir aristas entre SCCs en el DAG resultante.

Trampas y errores frecuentes.

- `inStack[u]` debe chequearse antes de usar `low[v] = min(low[v], disc[u])` en back-edges.

Codigo. Ver `cpp.tex`, seccion *Tarjan SCC*.

5.3 Matching

5.3.1 Kuhn (bipartite matching)

Idea intuitiva. Encuentra un **matching maximo** en un grafo bipartito en $O(VE)$. Para cada nodo u en L , intentar encontrar un vecino v en R que este libre o cuyo match se pueda “reorganizar” recursivamente. Si se encuentra, matchear $u - v$.

Propiedades clave (invariantes).

- `matchR[v]` = el nodo de L matcheado a v (o -1 si libre).
- `vis[]` se resetea **cada intento** de u .
- Si el matching es perfecto, $|L| = |R|$ = tamano del matching.

Complejidad.

- $O(VE)$ en el peor caso.

Donde tocar para modificar.

- **Matching bipartito minimo (min vertex cover):** Konig’s theorem.
- **Matching general:** blossom algorithm (no incluido).
- **Anadir capacidades:** si los nodos de R tienen capacidad > 1 , partir cada uno en capacidad copias.

Trampas y errores frecuentes.

- Olvidar resetear `vis` por intento; eso es lo que evita “ciclos infinitos” en la recursion.

Codigo. Ver `cpp.tex`, seccion *Kuhn (bipartite matching)*.

5.3.2 Hopcroft-Karp

Idea intuitiva. Matching bipartito maximo en $O(\sqrt{VE})$. Algoritmo en **fases:** en cada fase, hacer BFS para encontrar los **caminos de aumento mas cortos** desde nodos libres de L ; luego DFS solo por esos caminos. Cada fase aumenta el matching en ≥ 1 y toma $O(E)$.

Propiedades clave (invariantes).

- `dist[u]` = distancia en niveles desde un nodo libre de L .
- BFS termina cuando no hay mas camino a un nodo libre de R .
- DFS respeta los niveles (solo sigue `dist[matchV[v]] == dist[u] + 1`).

Complejidad.

- $O(\sqrt{VE})$.

Donde tocar para modificar.

- **Reconstruir el matching:** ya viene en `matchU/matchV`.
- **Aniadir pesos:** Hungarian algorithm.
- **Densidad alta:** si $E \approx V^2$, sigue siendo util pero verificar constantes.

Trampas y errores frecuentes.

- `dist[matchV[v]] == dist[u] + 1` (no `==` a secas); sin esto, el DFS no respeta los niveles y se pierde la complejidad.

Codigo. Ver `cpp.tex`, seccion *Hopcroft-Karp*.

5.4 Basicos

5.4.1 DFS / BFS / componentes

Idea intuitiva. **DFS** (Depth-First Search) y **BFS** (Breadth-First Search) son los dos recorridos basicos de grafos. DFS usa una pila (o recursion) y explora tan profundo como puede antes de retroceder; BFS usa una cola y explora por “niveles” de distancia. La cantidad de **componentes conexas** se cuenta iniciando un BFS/DFS desde cada nodo no visitado.

Propiedades clave (invariantes).

- BFS da la **distancia en aristas** mas corta desde el origen.
- DFS da un **orden topologico** (si se anaden al final).
- Ambos son $O(V + E)$.
- Un grafo con C componentes conexas satisface $\sum \text{subtree_size}_v = V$ y $\sum E_v = 2E$.

Complejidad.

- $O(V + E)$ tiempo, $O(V)$ memoria.

Donde tocar para modificar.

- **Aniadir pesos:** si las aristas tienen peso y quieres shortest path, BFS ya no sirve; usar Dijkstra.
- **Grafo dirigido:** en DFS, aniadir `check if (state[u] == 1)` para detectar ciclos.
- **Bipartite check:** BFS alternando colores (ver el siguiente modulo).
- **Componentes fuertemente conexas (dirigidos):** usar Tarjan o Kosaraju.

Trampas y errores frecuentes.

- Stack overflow con DFS recursivo para $V > 10^5$; usar version iterativa con pila explicita.

Codigo. Ver `cpp.tex`, seccion *DFS / BFS / componentes*.

5.4.2 Topological sort (Kahn + DFS)

Idea intuitiva. Un **orden topologico** de un DAG es una permutacion de los vertices tal que toda arista $u \rightarrow v$ tiene u antes que v . Dos algoritmos clasicos:

- **Kahn:** BFS sobre nodos con `indeg == 0`; al “consumir” un nodo, decrementar el `indeg` de sus vecinos; los que lleguen a 0 se encolan.
- **DFS:** hacer DFS; al terminar un nodo (estado = 2), aniadirlo al final; el orden es el reverso.

Propiedades clave (invariantes).

- Existe sii el grafo es un DAG.
- Si al final `order.size() < V`, hay un ciclo.
- Kahn da el orden “natural” (los que pueden ir primero); DFS da el orden “al revés” (los que pueden ir al final primero).

Complejidad.

- $O(V + E)$ ambos.

Donde tocar para modificar.

- **Detectar ciclos:** comparar `order.size()` con V .
- **Orden lexicografico minimo:** usar priority queue en Kahn.
- **Topological sort con pesos:** combinar con shortest path en DAG (un solo pase en orden topologico).

Trampas y errores frecuentes.

- En el snippet, Kahn retorna orden vacio si hay ciclo (util para detectar).
- En DFS, no olvidar `reverse(order)`.

Codigo. Ver `cpp.tex`, seccion *Topological sort (Kahn + DFS)*.

5.4.3 Bipartite check + 2-coloring

Idea intuitiva. Un grafo es **bipartito** si sus vertices se pueden dividir en dos conjuntos L y R tales que toda arista va de L a R . Esto equivale a decir que se puede **2-colorear** sin que dos vertices adyacentes tengan el mismo color. BFS alternando colores detecta esto: si en algun momento dos adyacentes tienen el mismo color, no es bipartito.

Propiedades clave (invariantes).

- Grafo bipartito $\Leftrightarrow$ sin ciclos impares.
- Conexo: si un componente no es bipartito, todo el grafo no lo es.
- Si hay **multiple componentes**, cada uno se colorea independientemente.

Complejidad.

- $O(V + E)$.

Donde tocar para modificar.

- **Colorear desde multiples fuentes:** BFS desde cada nodo no coloreado.
- **Bipartito + matching:** Hopcroft-Karp.
- **Grafo bipartito ponderado:** Hungarian algorithm.

Trampas y errores frecuentes.

- Verificar **todos** los componentes, no solo el primero.

Codigo. Ver `cpp.tex`, seccion *Bipartite check + 2-coloring*.

5.4.4 0-1 BFS

Idea intuitiva. Variante de BFS para grafos con **pesos 0 o 1**. Usa un **deque**: aristas de peso 0 se encolan al frente (`push_front`), aristas de peso 1 al final (`push_back`). Así, la primera vez que se extrae un nodo, su distancia es la definitiva, igual que en Dijkstra pero en $O(V + E)$.

Propiedades clave (invariantes).

- Asume pesos **solo** 0 o 1.
- Mejor caso que Dijkstra (sin factor $\log V$).
- No funciona con pesos ≥ 2 .

Complejidad.

- $O(V + E)$.

Donde tocar para modificar.

- **Pesos en $[0, k]$ small:** usar small k-ary BFS o Dijkstra.
- **Reconstruir camino:** añadir `parent[]`.

Trampas y errores frecuentes.

- Si los pesos son otro rango, el algoritmo da respuestas incorrectas. Verificar que las aristas son solo 0/1.

Código. Ver `cpp.tex`, sección *0-1 BFS*.

5.5 MST

5.5.1 Kruskal

Idea intuitiva. Encuentra el **arbol de expansion minima** (MST) en $O(E \log E)$. Ordena las aristas por peso ascendente y las añade al MST si no forman ciclo (chequear con DSU). El arbol tiene exactamente $V - 1$ aristas.

Propiedades clave (invariantes).

- Algoritmo **greedy** correcto (Cut Property).
- Funciona en grafos **no conexos**; produce un **minimum spanning forest**.
- Si las aristas tienen pesos únicos, el MST es único.

Complejidad.

- $O(E \log E)$ (ordenamiento) o $O(E \log V)$ con DSU optimizations.

Donde tocar para modificar.

- **Second best MST:** enumerar aristas del MST, sacar una, añadir una no-MST; recalcular.
- **Kruskal randomizado:** si el grafo es casi-completo, agregar shuffle.
- **Añadir info al DSU:** guardar el peso máximo en el camino, etc.

Trampas y errores frecuentes.

- El DSU debe estar limpio para cada invocación.
- Si las aristas tienen pesos negativos, Kruskal sigue funcionando.

Código. Ver `cpp.tex`, sección *Kruskal*.

5.5.2 Prim (priority queue)

Idea intuitiva. Otra forma de calcular el MST: empezar desde un nodo y , en cada paso, añadir la arista de menor peso que conecta el conjunto construido con el resto. Usando priority queue, es $O((V + E) \log V)$.

Propiedades clave (invariantes).

- Similar a Dijkstra pero sin acumular distancias, solo eligiendo mínimo por arista.
- Funciona mejor que Kruskal en **grafos densos** ($E \approx V^2$).
- Si el grafo es desconexo, genera el bosque (solo llega a los alcanzables).

Complejidad.

- $O((V + E) \log V)$.

Donde tocar para modificar.

- **Prim con matriz de adyacencia:** $O(V^2)$ sin heap, útil en grafos densos pequeños.
- **Reconstruir el MST:** añadir `parent[]`.

Trampas y errores frecuentes.

- El `if (used[v]) continue;` es crucial: si el nodo ya fue procesado, saltar.

Código. Ver `cpp.tex`, sección *Prim (priority queue)*.

5.6 Flujos

5.6.1 Edmonds-Karp (max flow)

Idea intuitiva. Implementación BFS del algoritmo de **Ford-Fulkerson**. En cada iteración, hacer BFS desde s hasta t siguiendo aristas con capacidad residual > 0 ; si se llega a t , se encontró un **camino de aumento**. Enviar el mínimo de las capacidades en el camino; actualizar capacidades residuales.

Propiedades clave (invariantes).

- BFS garantiza caminos de aumento más cortos.
- Capacidad residual: forward edge baja, backward edge sube.
- Complejidad $O(VE^2)$.

Complejidad.

- $O(VE^2)$.

Donde tocar para modificar.

- **Grafo bipartito:** $O(\sqrt{V}E)$ (equivalente a Hopcroft-Karp).
- **Dinic** es preferible en la mayoría de los casos.
- **Capacidades doubles:** usar tolerancia para comparación.

Trampas y errores frecuentes.

- Las capacidades deben ser > 0 para entrar a la cola (no ≥ 0); sino, ciclos infinitos.

Código. Ver `cpp.tex`, sección *Edmonds-Karp (max flow)*.

5.6.2 Dinic (max flow)

Idea intuitiva. Algoritmo de flujo mas eficiente que Edmonds-Karp: combina **BFS para level graph** con **DFS bloqueante** (multiple augmenting paths en una sola pasada). Cada fase cuesta $O(E)$ y hay $O(V)$ fases en el peor caso; para bipartitos, $O(\sqrt{V}E)$.

Propiedades clave (invariantes).

- `level[v]` = distancia desde s en el level graph (solo aristas con $\text{cap} > 0$).
- `it[v]` = iterador actual para evitar reprocesar aristas.
- DFS solo sigue aristas con `level[e.to] == level[v] + 1`.
- Back-edges tienen `rev` apuntando a la forward edge original.

Complejidad.

- $O(V^2E)$ general, $O(\sqrt{V}E)$ para bipartitos.

Donde tocar para modificar.

- **Dinic con scaling:** anadir factor de escala para capacidades grandes.
- **Lower bounds:** anadir un super-source/sink para forzar flujo minimo.
- **Matching bipartito:** el grafo se construye como source $\rightarrow L \rightarrow R \rightarrow$ sink, todos con capacidad 1.

Trampas y errores frecuentes.

- `it[v]` debe **incrementarse** en cada iteracion; sino, el DFS se queda atascado en la misma arista.

Codigo. Ver `cpp.tex`, seccion *Dinic (max flow)*.

5.6.3 Min-Cost Max-Flow

Idea intuitiva. Encuentra el flujo maximo con **costo minimo** (asumiendo costos no negativos). Usa **SPFA** (o Dijkstra con potentials) para encontrar el shortest path en el **costo** desde s hasta t considerando capacidades residuales. Cada augmenting path aporta su costo $\times$ flujo.

Propiedades clave (invariantes).

- `pot[]` = potentials para evitar aristas negativas (Johnson's trick).
- `dist[v]` = costo minimo desde s con potentials.
- Solo se aumentan aristas con $\text{cap} > 0$ (forward) y costo es **+cost** (forward) o **-cost** (backward).

Complejidad.

- $O(F \cdot VE)$ con SPFA, $O(F \cdot E \log V)$ con Dijkstra + potentials.

Donde tocar para modificar.

- **Dijkstra + potentials:** si los costos son enteros no negativos, esto es mas estable.
- **Costos negativos en aristas:** requieren Bellman-Ford inicial o reorden.
- **Flujo minimo con costo:** encontrar el minimo flujo posible y su costo.

Trampas y errores frecuentes.

- El `pot[]` se actualiza **al final** de cada iteracion (no durante).
- Las back-edges tienen **costo negativo**, esencial para corregir elecciones previas.

Codigo. Ver `cpp.tex`, seccion *Min-Cost Max-Flow*.

5.7 Especiales

5.7.1 2-SAT (implication graph + SCC)

Idea intuitiva. Resuelve formulas booleanas en **CNF** con clausulas de **2 literales**: $(l_1 \vee l_2)$. Cada variable x se representa con dos nodos: x (verdadero) y $\neg x$ (falso). Cada clausula $l_1 \vee l_2$ se descompone en dos implicaciones: $\neg l_1 \rightarrow l_2$ y $\neg l_2 \rightarrow l_1$. Si en el grafo de implicaciones $\neg x$ y x estan en la misma SCC, la formula es **UNSAT**.

Propiedades clave (invariantes).

- Cada literal es un nodo; total $2n$ nodos.
- Kosaraju/Tarjan dan las SCCs.
- Si SAT: asignar $x =$ (topological order del condensation).
- Clausulas `setTrue(x)` se modelan como `implies(!x, x)`.

Complejidad.

- $O(V + E) = O(N + M)$ con Kosaraju o Tarjan.

Donde tocar para modificar.

- **Forzar variables:** anadir `setTrue` / `setFalse`.
- **3-SAT:** NP-completo; no usar este algoritmo.
- **Contar soluciones:** las SCCs condensation dan una formula para contar; multiplicar las formas de asignar variables en cada componente no-trivial.

Trampas y errores frecuentes.

- La asignacion usa el **orden topologico inverso** del condensation: `val[i] = comp[2i] < comp[2i+1]`.

Codigo. Ver `cpp.tex`, seccion *2-SAT (implication graph + SCC)*.

5.7.2 Euler tour / Hierholzer (camino y circuito)

Idea intuitiva. Encuentra un **camino o circuito euleriano** (que usa cada arista exactamente una vez). **Condiciones:**

- **No dirigido:** todos los grados pares (circuito) o exactamente 2 impares (camino, empieza en uno, termina en el otro). Ademas, conexo sobre el subgrafo con aristas.
- **Dirigido:** $\text{indeg}(v) = \text{outdeg}(v)$ para todos (circuito) o exactamente un nodo con $\text{outdeg} = \text{indeg} + 1$ y otro con $\text{indeg} = \text{outdeg} + 1$ (camino).

Hierholzer : elegir un ciclo desde el inicio, mientras haya ciclos “laterales” desde un nodo del camino, mergearlos. Implementacion con pila: avanzar por aristas, al no poder mas anadir a la respuesta y backtrack.

Propiedades clave (invariantes).

- Multigrafos permitidos; las aristas se marcan como usadas.
- No dirigido: cada arista aparece **dos veces** en `adj` (ida y vuelta).
- La construccion del path con ids consistentes es importante.

Complejidad.

- $O(V + E)$.

Donde tocar para modificar.

- **Reconstruir con multigrafo:** añadir ids consistentes antes del algoritmo.
- **Aplicar a “de Bruijn” o problemas de strings:** modelar como multigrafo y buscar Euler.
- **Dirigido vs no dirigido:** el snippet tiene ambas versiones.

Trampas y errores frecuentes.

- Si el grafo no es conexo (sobre las aristas), no existe euleriano. Verificar primero.

Codigo. Ver `cpp.tex`, seccion *Euler tour / Hierholzer (camino y circuito)*.

5.7.3 Hamiltonian path / cycle (bitmask DP + backtracking)

Idea intuitiva. Un **camino hamiltoniano** visita cada vertice **exactamente una vez**. **Determinar** si existe es NP-completo en general, pero con $V \leq 20$ se puede hacer DP con bitmask en $O(V^2 \cdot 2^V)$. **Teoremas suficientes:** **Dirac** (grado $\geq n/2$) y **Ore** ($\deg(u) + \deg(v) \geq n$ para todo par no adyacente).

Propiedades clave (invariantes).

- DP: $dp[mask][v] = \text{true}$ si hay un camino que visita exactamente `mask` y termina en v .
- TSP: variante con pesos minimos (Hamilton cycle minimo).
- Reconstruccion: guardar `par[mask][v]` (predecesor).

Complejidad.

- $O(V^2 \cdot 2^V)$ tiempo, $O(V \cdot 2^V)$ memoria.

Donde tocar para modificar.

- **TSP asimetrico:** añadir pesos $w[v][u]$ (no necesariamente simetricos).
- **Hamilton con inicio obligatorio:** fijar `src` en el DP inicial.
- **Bitmask DP mas compacto:** usar `long long` como mascara si $V \leq 64$.
- **TSP optimo aproximado:** MST, 2-opt, simulated annealing.

Trampas y errores frecuentes.

- TSP asume grafo **completo** (con LINF para aristas faltantes); si no, añadir chequeo.

Codigo. Ver `cpp.tex`, seccion *Hamiltonian path / cycle (bitmask DP + backtracking)*.

6 DP

6.0.1 Knapsack 0/1

Idea intuitiva. Dado un conjunto de N objetos con peso w_i y valor v_i , maximizar $\sum v_i$ con la restriccion $\sum w_i \leq W$. La recurrencia clasica:

$$dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w_i] + v_i) \quad \text{si } j \geq w_i.$$

La optimizacion clave: la DP es 1-dimensional si iteramos j **de mayor a menor** (asi no usamos el objeto i dos veces).

Propiedades clave (invariantes).

- $dp[j]$ al final = maximo valor con peso $\leq j$.
- El bucle inverso `for (j = W; j >= w[i]; j--)` es lo que distingue 0/1 de unbounded.
- Solucion: maximo valor con cualquier peso $\leq W$.

Complejidad.

- $O(N \cdot W)$ tiempo, $O(W)$ memoria.

Donde tocar para modificar.

- **Reconstruir la seleccion:** anadir `take[i][j]` (booleano o bool) en una version 2D.
- **Items con peso 0:** edge case; anadir guard.
- **Knapsack con restricciones de grupos:** “elegir a lo sumo 1 de cada grupo” se modela como 0/1 con un peso extra.
- **Tamano de W :** si $W \leq 10^9$ pero N es chico, usar meet-in-the-middle.

Trampas y errores frecuentes.

- Overflow en `dp[j - w[i]] + v[i]`; usar `long long` si los valores son grandes.
- El bucle **debe** ir de mayor a menor.

Codigo. Ver `cpp.tex`, seccion *Knapsack 0/1*.

6.0.2 Knapsack unbounded

Idea intuitiva. Igual que Knapsack 0/1 pero **cada item se puede tomar multiples veces**. La recurrencia es similar:

$$dp[j] = \max(dp[j], dp[j - w_i] + v_i),$$

pero ahora el bucle va de **menor a mayor** (porque tomar el item i no afecta corridas futuras del mismo item).

Propiedades clave (invariantes).

- El bucle `for (j = w[i]; j <= W; j++)` es lo que distingue unbounded.
- A diferencia de 0/1, no hay riesgo de “usar dos veces el mismo item”.

Complejidad.

- $O(N \cdot W)$ tiempo, $O(W)$ memoria.

Donde tocar para modificar.

- **Reconstruir cuantos de cada item:** anadir `cnt[i]` y backtrack.
- **Cambiar moneda minima por maxima:** el snippet maximiza valor; para minimizar monedas, cambiar a `min` con inicializacion en `INF`.

Trampas y errores frecuentes.

- Misma que Knapsack 0/1 con overflow.

Codigo. Ver `cpp.tex`, seccion *Knapsack unbounded*.

6.0.3 Knapsack meet-in-the-middle

Idea intuitiva. Para $N \leq 40$ y W moderado, no se puede hacer $O(NW)$ directamente. Partir en dos mitades, enumerar todos los $2^{N/2}$ subconjuntos de cada mitad, y combinar. Para cada subconjunto de la segunda mitad con peso sw_2 , encontrar el maximo valor de la primera mitad con peso $\leq W - sw_2$.

Propiedades clave (invariantes).

- Hay $2^{N/2}$ subconjuntos por mitad; total $O(2^{N/2})$.
- `sums` guarda los valores de subconjuntos de la primera mitad ordenados (o preprocesados).
- Para cada subconjunto de la segunda mitad, busqueda lineal o binaria en `sums`.

Complejidad.

- $O(2^{N/2})$ tiempo y memoria (mejor que $O(NW)$ cuando $W > 2^{N/2}$).

Donde tocar para modificar.

- **Tamano mayor:** si N es mayor pero W es chico, conviene iterar por peso, no por subconjuntos.
- **Varias mochilas:** adaptar para multiples W .
- **Optimizar el “combinado”:** usar `upper_bound` para encontrar el maximo valor con peso $\leq$ rem.

Trampas y errores frecuentes.

- N impar: una mitad tiene $N/2 + 1$ items.
- Overflow al sumar pesos/valores en `long long`.

Codigo. Ver `cpp.tex`, seccion *Knapsack meet-in-the-middle*.

6.0.4 LIS $O(n \log n)$

Idea intuitiva. La **Longest Increasing Subsequence** se calcula en $O(n \log n)$ con el truco del **patience sorting**: mantener un arreglo `dp` donde `dp[1en]` es el minimo ultimo valor de una IS de largo `1en`. Para cada x , encontrar el primer `dp[1en] >= x` y reemplazarlo. El largo final es la LIS.

Propiedades clave (invariantes).

- `dp` es estrictamente creciente.
- `lower_bound` da LIS estricta; `upper_bound` da no-estricta ($\leq$).
- El algoritmo es **online**: procesa elementos en orden.

Complejidad.

- $O(n \log n)$.

Donde tocar para modificar.

- **Reconstruir la LIS:** anadir `parent[]` y `posInDp[]` para backtrack.
- **Contar LIS:** DP clasica $O(n^2)$ o segment tree sobre `dp`.
- **LDS / Longest non-increasing:** invertir la condicion en `lower_bound` o multiplicar por -1.
- **Longest bitonic subsequence:** combinar LIS desde izq y desde der.

Trampas y errores frecuentes.

- `lower_bound` vs `upper_bound`: si tu comparacion es $\leq$ vs $<$, cambia.

Codigo. Ver `cpp.tex`, seccion *LIS* $O(n \log n)$.

6.0.5 LCS / Edit distance

Idea intuitiva. **LCS** (Longest Common Subsequence): para dos strings a, b , $dp[i][j]$ = LCS de $a[0..i]$ y $b[0..j]$.
Recurrencia:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{si } a[i-1] = b[j-1] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{si no} \end{cases}$$

Edit distance (Levenshtein): numero minimo de operaciones (insert, delete, replace) para convertir a en b .
Recurrencia similar con un +1 adicional.

Propiedades clave (invariantes).

- LCS y Edit Distance son DP $O(nm)$.
- Memoria se puede reducir a $O(\min(n, m))$ con rolling array.
- Edit distance con peso por operacion distinto: ajustar el +1 por el peso.

Complejidad.

- $O(NM)$ tiempo y memoria.

Donde tocar para modificar.

- **Reconstruir la LCS / las operaciones:** anadir `prev[i][j]` para backtrack.
- **LCS de varios strings:** NP-hard para ≥ 3 strings; en la practica se hace con bitmask.
- **Edit distance con costos distintos:** cambiar el +1 por el costo apropiado.
- **Rolling array:** si memoria es problema, mantener solo 2 filas.

Trampas y errores frecuentes.

- Indices en la recurrencia: `a[i-1]` y `b[j-1]` (no `a[i]`).

Codigo. Ver `cpp.tex`, seccion *LCS / Edit distance*.

6.0.6 Bitmask DP over subsets

Idea intuitiva. Cuando el estado es un **subconjunto** de $\{0, \dots, N-1\}$, se representa como una mascara de N bits. Para iterar sobre todos los subconjuntos de `mask`:

for (sub = mask; sub; sub = (sub - 1) & mask) { ... }

Para iterar sobre todos los superconjuntos de `mask` en $[0, U)$:

for (sup = mask; sup < U; sup = (sup + 1) | mask) { ... }

Propiedades clave (invariantes).

- Iterar subsets: $O(2^k)$ para una mascara con k bits.
- `sub & mask == sub` siempre; `mask - sub` no siempre da un subconjunto.
- `__builtin_popcount(mask)` cuenta los bits.

Complejidad.

- $O(N \cdot 2^N)$ para una DP estandar sobre todos los subsets.

Donde tocar para modificar.

- **TSP / Hamiltonian path:** bitmask DP (ver tambien *Hamiltonian path* en `cpp.tex` seccion Grafos).
- **Set cover:** iterar subsets.
- **DP con supermask:** si el estado incluye “todos los elementos que **no** hemos usado”, usar supermask iteration.

Trampas y errores frecuentes.

- El orden de iteracion de subsets de `mask` es decreciente (de `mask` a 0).
- `mask = 0` no tiene subsets no triviales; el bucle lo saltea correctamente.

Codigo. Ver `cpp.tex`, seccion *Bitmask DP over subsets*.

6.0.7 SOS DP

Idea intuitiva. Para calcular, para cada `mask`, la suma de $f(\text{submask})$ sobre todos los $\text{submask} \subseteq \text{mask}$:

$$dp[\text{mask}] = \sum_{\text{sub} \subseteq \text{mask}} f(\text{sub}).$$

Se computa en $O(N \cdot 2^N)$ mediante un loop por bit: para cada i , sumar a $dp[\text{mask}]$ la contribucion de $dp[\text{mask} \oplus (1 \ll i)]$ cuando mask tiene el bit i .

Propiedades clave (invariantes).

- Inicializar $dp = f$.
- Para cada bit i : `if (mask & (1 << i)) dp[mask] += dp[mask ^ (1 << i)]`.
- Variantes: supersets DP (sumar sobre superconjuntos).

Complejidad.

- $O(N \cdot 2^N)$.

Donde tocar para modificar.

- **Max/min en lugar de suma:** cambiar `+=` por la operacion correspondiente.
- **Supersets DP:** iterar `mask` de 0 a 2^N y anadir `dp[mask] += dp[mask | (1<<i)]`.
- **XOR convolution** (subset convolution): version mas avanzada.

Trampas y errores frecuentes.

- Si N es 30 o mas, 2^N no entra en memoria; usar trucos o reducir N .

Codigo. Ver `cpp.tex`, seccion *SOS DP*.

6.0.8 Digit DP template

Idea intuitiva. Contar numeros en $[0, N]$ que cumplen una propiedad sobre sus digitos. La idea: DP por posicion, manteniendo:

- **pos:** posicion actual (de izquierda a derecha).
- **tight:** si los digitos ya colocados son exactamente los de N (limite superior estricto).
- **state:** estado adicional (digitos usados, suma, etc.).

Propiedades clave (invariantes).

- Si `tight` es true, el siguiente dígito está acotado por el de N ; si no, puede ser 0-9.
- Memoria: $O(D \cdot 2 \cdot \text{states})$.
- Caso base: si `pos == D`, devolver 1 si el estado cumple la propiedad, sino 0.

Complejidad.

- $O(D \cdot 10 \cdot \text{states})$ donde D es el número de dígitos.

Donde tocar para modificar.

- **Cambiar la propiedad:** modificar `updateState` y la condición base.
- **Múltiples estados:** añadir más dimensiones al `memo`.
- **Leading zeros:** añadir un flag `started` para distinguir números con menos dígitos.

Trampas y errores frecuentes.

- Olvidar resetear `memo` entre queries.
- Si la cantidad de estados es grande, el `memo[pos][tight][state]` puede ser muy grande; usar `map`.

Código. Ver `cpp.tex`, sección *Digit DP template*.

6.0.9 Tree DP (reroot)

Idea intuitiva. Calcular alguna propiedad asumiendo que **cada nodo es la raíz** del árbol. La técnica clásica de **rerooting** en dos pasadas:

- **DFS 1:** desde una raíz arbitraria, calcular `dp[v]` para el subárbol de v .
- **DFS 2:** para cada hijo u de v , calcular `up[u]` = respuesta si la raíz fuera u .

Propiedades clave (invariantes).

- `dp[v]` depende solo del subárbol de v (con la raíz original).
- `up[u]` se calcula de manera que cuando la raíz es u , “todo lo que estaba en v se ajusta”.
- Fórmula para reroot depende del problema; el snippet usa el ejemplo “suma de distancias”.

Complejidad.

- $O(N)$ para ambas pasadas.

Donde tocar para modificar.

- **Cambiar la cantidad a calcular:** modificar la operación en `dfs1` y la fórmula en `dfs2`.
- **Árbol con pesos:** añadir `weights[v]` y ajustar.
- **Re-root en árboles con ciclos:** requiere otra técnica (DSU on tree).

Trampas y errores frecuentes.

- La fórmula de `up[u]` debe sumar y restar correctamente las contribuciones de u y del resto.

Código. Ver `cpp.tex`, sección *Tree DP (reroot)*.

7 Geometria

7.1 Puntos y segmentos

7.1.1 Point + cross + dot

Idea intuitiva. La estructura `Point` representa un punto en 2D con coordenadas enteras. Las dos operaciones vectoriales clave son:

- **Cross product** $P \times Q = P_x Q_y - P_y Q_x$. Da el “area con signo” del paralelogramo formado por P y Q . Usado para orientacion.
- **Dot product** $P \cdot Q = P_x Q_x + P_y Q_y$. Da el producto de las longitudes por el coseno del angulo. Usado para proyeccion.

Propiedades clave (invariantes).

- `cross(a, b) > 0` si b esta a la izquierda de a (sistema de coordenadas usual).
- `cross(a, b) = 0` si son colineales.
- `dot(a, b) > 0` si el angulo es agudo.
- El snippet define `cross` respecto a `this`, util para orientacion de 3 puntos.

Complejidad.

- $O(1)$ por operacion.

Donde tocar para modificar.

- **Coordenadas doubles:** cambiar `int` por `double` o `long double`. Cuidado con la precision.
- **3D:** anadir `z` y adaptar `cross/dot`.
- **Operadores adicionales:** anadir `operator+`, `operator-`, `operator*` (escalar).
- **Distancia al cuadrado:** anadir `len2()` que retorna $x^2 + y^2$.

Trampas y errores frecuentes.

- Cross product **no** es conmutativo; $P \times Q = -Q \times P$.
- Overflow con coordenadas grandes ($|x|, |y| > 10^4$) si se hace `x * y`.

Codigo. Ver `cpp.tex`, seccion *Point + cross + dot*.

7.1.2 Interseccion de segmentos

Idea intuitiva. Dados dos segmentos $[a, b]$ y $[c, d]$, determinar si se intersectan. La idea clasica:

- Verificar orientaciones: a, b, c y a, b, d deben estar en lados opuestos de la recta ab ; igual para cd y ab .
- Caso colineal: si las rectas son paralelas (`cross` = 0) y los segmentos son colineales, verificar que los bounding boxes se intersectan.

Propiedades clave (invariantes).

- Funciona con segmentos en cualquier orientacion.
- Caso colineal: requiere check adicional de bounding box.
- Para intersectar en un **punto** especifico: anadir division con cuidado de precision.

Complejidad.

- $O(1)$.

Donde tocar para modificar.

- **Interseccion con punto:** anadir la division despues de verificar orientacion.
- **Interseccion rayo-segmento:** ajustar la primera condicion.
- **Poligonos convexos:** usar separacion de ejes (SAT).

Trampas y errores frecuentes.

- Si los segmentos tocan solo en un extremo, el snippet lo considera interseccion (algunos problemas quieren estricto). Ajustar segun el problema.

Codigo. Ver `cpp.tex`, seccion *Interseccion de segmentos*.

7.1.3 Contains colineal

Idea intuitiva. Verifica si un punto c esta sobre el segmento $[a, b]$ (asumiendo que ya son colineales). Si `cross(a, b, c) = 0` y c esta dentro del bounding box de $[a, b]$, entonces c esta en el segmento.

Propiedades clave (invariantes).

- Asume colinealidad (no la verifica mas alla de `cross == 0`).
- Usa comparaciones con `<=` para extremos **inclusivos**.

Complejidad.

- $O(1)$.

Donde tocar para modificar.

- **Contener estricto:** cambiar `<=` a `<` para excluir extremos.
- **Coordenadas doubles:** usar tolerancia (`eps`).

Trampas y errores frecuentes.

- No usar este snippet para puntos **no colineales**; siempre verificar primero la colinealidad.

Codigo. Ver `cpp.tex`, seccion *Contains colineal*.

7.2 Poligonos

7.2.1 Convex Hull (monotono)

Idea intuitiva. Algoritmo de **Andrew's monotone chain**: ordenar puntos por (x, y) , luego construir la hull inferior y superior por separado. Para cada punto, anadirlo al hull y, mientras el ultimo giro no sea "counter-clockwise" (`cross > 0`), ppear.

Propiedades clave (invariantes).

- Ordena los puntos en $O(n \log n)$.
- Construye la hull en $O(n)$ despues.
- Output: poligono convexo en counter-clockwise, sin punto final duplicado.
- El `<= 0` incluye colineales; usar `< 0` para excluirlos.

Complejidad.

- $O(n \log n)$.

Donde tocar para modificar.

- **Colineales:** cambiar ≤ 0 por < 0 para excluirlos del hull.
- **Puntos repetidos:** `unique` despues de sort.
- **Hull 3D:** gift wrapping es $O(n^2)$; convex hull 3D es mas complejo.

Trampas y errores frecuentes.

- El `pop_back` y el `reverse` son esenciales para no duplicar el ultimo punto.

Codigo. Ver `cpp.tex`, seccion *Convex Hull (monotono)*.

7.2.2 Polygon Area

Idea intuitiva. El area (con signo) de un poligono se calcula con la **Shoelace formula**: $A = \frac{1}{2} \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i)$, donde el indice es modulo n . El snippet retorna el **doble** del area (en valor absoluto).

Propiedades clave (invariantes).

- El signo indica la orientacion: positivo si counter-clockwise.
- Para el doble: no dividir por 2 si no es necesario (evita fracciones).

Complejidad.

- $O(n)$.

Donde tocar para modificar.

- **Area exacta:** dividir por 2 (con cuidado si da fraccion; usar `double`).
- **Area firmada:** quitar `abs` y dejar el signo.
- **Punto dentro de poligono convexo:** con la hull, chequear que esta a un mismo lado de todas las aristas.

Trampas y errores frecuentes.

- Si los puntos no estan en orden (CCW o CW), el area firmada puede ser incorrecta. Para signed area consistente, usar siempre el mismo orden.

Codigo. Ver `cpp.tex`, seccion *Polygon Area*.

7.2.3 Distance point-segment / point-line

Idea intuitiva. Distancia minima de un punto p a un segmento $[a, b]$ o a la recta que pasa por a y b .

- **Linea:** $d = \frac{|ab \times ap|}{|ab|}$.
- **Segmento:** si la proyeccion de p sobre ab cae fuera del segmento, la distancia es al extremo mas cercano.

Propiedades clave (invariantes).

- `dist2PointLine` retorna distancia **al cuadrado**.
- `dist2PointSegment` distingue 3 casos: p antes de a , despues de b , o proyectado sobre el segmento.
- Si `len2 = 0` (degenerate), la distancia es $|ap|$ o $|bp|$.

Complejidad.

- $O(1)$.

Donde tocar para modificar.

- **Distancia (no al cuadrado):** añadir `sqrt` al final.
- **Coordenadas doubles:** añadir `long double` y tolerancia.
- **3D:** añadir coordenada z y adaptar.

Trampas y errores frecuentes.

- Overflow en `area2 * area2` si el area es grande; `long long` alcanza para $|x|, |y| \leq 10^9$ pero no mas.

Codigo. Ver `cpp.tex`, seccion *Distance point-segment / point-line*.

7.2.4 Projection of point onto line

Idea intuitiva. Proyeccion ortogonal de p sobre la recta que pasa por a y b . El parametro t tal que la proyeccion es $a + t \cdot (b - a)$ es:

$$t = \frac{(p - a) \cdot (b - a)}{|b - a|^2}.$$

Propiedades clave (invariantes).

- Si $t \in [0, 1]$, la proyeccion cae sobre el **segmento**; si no, cae sobre la extension.
- Retorna `double` (puede tener precision issues).

Complejidad.

- $O(1)$.

Donde tocar para modificar.

- **Proyeccion sobre segmento:** clipar t a $[0, 1]$ antes de calcular el punto.
- **Distancia al segmento via proyeccion:** combinar con el modulo anterior.

Trampas y errores frecuentes.

- Si $a = b$ ($|ab| = 0$), la proyeccion no esta definida; añadir guard.

Codigo. Ver `cpp.tex`, seccion *Projection of point onto line*.

7.2.5 Point in polygon (ray casting)

Idea intuitiva. Algoritmo de **ray casting**: contar cuantas veces un rayo horizontal desde p cruza los bordes del poligono. Si es impar, p esta dentro; si par, fuera. El snippet implementa la version clasica recorriendo aristas.

Propiedades clave (invariantes).

- Funciona con poligonos simples (no necesariamente convexos).
- Las intersecciones en vertices cuentan especialmente.
- Para poligonos con **huecos**, sigue funcionando.

Complejidad.

- $O(n)$ por query.

Donde tocar para modificar.

- **Winding number:** version alternativa que cuenta el numero de vueltas.
- **Poligono convexo:** check lineal contra las aristas (mas rapido).
- **Multiples queries:** precomputar o usar segment tree sobre las aristas.

Trampas y errores frecuentes.

- El manejo del vertice (cuando el rayo pasa por un vertice) puede causar double-counting; el snippet lo evita con la condicion $(poly[i].y > p.y) \neq (poly[j].y > p.y)$.

Codigo. Ver `cpp.tex`, seccion *Point in polygon (ray casting)*.

7.2.6 Minkowski sum (convex polygons)

Idea intuitiva. La **suma de Minkowski** $A \oplus B$ de dos poligonos convexos es otro poligono convexo que representa $\{a + b : a \in A, b \in B\}$. Algoritmo: ordenar las aristas por angulo polar (CCW) y “mergearlas” como en merge sort; el resultado es un poligono CCW sin repetir el ultimo punto.

Propiedades clave (invariantes).

- Si A, B son convexos con n, m vertices, $A \oplus B$ tiene $\leq n + m$ vertices.
- Util para **detectar colision**: A colisiona con B si $0 \in A \oplus (-B)$.
- Asume poligonos en orden **counter-clockwise**.

Complejidad.

- $O(n + m)$.

Donde tocar para modificar.

- **Deteccion de colision:** anadir $(-B)$ (reflejar) y verificar si 0 esta en el resultado.
- **Poligonos no convexos:** la suma puede no ser poligono simple; algoritmo distinto.
- **Multiples queries:** precomputar las aristas una vez.

Trampas y errores frecuentes.

- Si los poligonos no son CCW, la suma es incorrecta. Normalizar antes.

Codigo. Ver `cpp.tex`, seccion *Minkowski sum (convex polygons)*.

7.3 Herramientas

7.3.1 Li Chao Tree (long long)

Idea intuitiva. Mantiene un **lower envelope** (o upper) de un conjunto de rectas $y = mx + b$ sobre un dominio $[X_{LO}, X_{HI}]$ y responde “minimo y en un x dado” en $O(\log X)$. La estructura es un segment tree donde cada nodo guarda la recta “mejor” para su segmento medio; al anadir una recta, se compara con la actual y se decide cual es mejor en cada mitad, descendiendo recursivamente.

Propiedades clave (invariantes).

- **Online:** cada `addLine` en $O(\log X)$.
- Solo insercion; no hay borrar (para borrar, usar Li Chao segmentado con reset).
- La recta guardada en un nodo es la “mejor” en su punto medio.

Complejidad.

- $O(\log X)$ por insercion y query, donde X es el tamaño del dominio.

Donde tocar para modificar.

- **Upper envelope en vez de lower:** invertir el signo ($>$ en vez de $<$).
- **Queries de segmento** (no full line): si la recta solo esta activa en un rango, añadir bandera **active**.
- **Eliminar rectas:** usar Li Chao segmentado (mas complejo).
- **Pendientes grandes:** si las pendientes tienen magnitud $> 10^9$, añadir `__int128` para evitar overflow.

Trampas y errores frecuentes.

- El parametro `lo == hi` debe retornar el valor del nodo (caso base).
- La comparacion `newLine.get(mid) < node->line.get(mid)` decide quien es mejor en el medio.

Codigo. Ver `cpp.tex`, seccion *Li Chao Tree (long long)*.

7.3.2 Sweep line template

Idea intuitiva. Patron de diseno para problemas geometricos donde se barre una linea sobre el plano y se mantiene una estructura de datos sobre el otro eje. Eventos (add, query, remove) se ordenan por la coordenada de barrido y se procesan secuencialmente.

Propiedades clave (invariantes).

- Eventos: tuplas (`x`, `type`, `id`).
- La estructura auxiliar (set, multiset, segtree) se mantiene sobre el otro eje.
- Procesar eventos en orden de x ascendente.

Complejidad.

- Depende del problema; tipico $O((N + E) \log N)$.

Donde tocar para modificar.

- **Aniadir/quitar segmentos:** añadir eventos add/remove al barrido.
- **Aniadir queries:** añadir eventos query.
- **Cambiar el eje de barrido:** si barres en y en vez de x , añadir comparador.

Trampas y errores frecuentes.

- El snippet es solo el **esqueleto**; los handlers **add/query/remove** se llenan por problema.
- Cuidado con eventos duplicados.

Codigo. Ver `cpp.tex`, seccion *Sweep line template*.

7.3.3 Closest pair of points

Idea intuitiva. Encuentra el par de puntos mas cercano en $O(n \log n)$ con **divide y venceras**. Algoritmo:

- Sort por x .
- Dividir en mitades; recursivo.
- Combinar: dado el minimo d de las dos mitades, strip = puntos en $[midX - d, midX + d]$; ordenar por y ; cada punto solo se compara con los siguientes hasta y difiera en $< d$ (maximo 7).

Propiedades clave (invariantes).

- Requiere ordenar por y despues (en el snippet se hace durante el merge).
- Strip tiene a lo sumo $O(n)$ puntos en total, pero la comparacion es $O(1)$ amortizada por punto.
- Resultado en `double`.

Complejidad.

- $O(n \log n)$.

Donde tocar para modificar.

- **Reconstruir el par:** aniadir indices en la recursion.
- **Distancia maxima:** cambiar `min` por `max` (mas facil; no requiere D&C).
- **Puntos duplicados:** aniadir check especial (distancia 0).

Trampas y errores frecuentes.

- El merge requiere ordenar por y ; el snippet lo hace in-place con un `tmp`.
- Si se hace en cada recursion, es $O(n \log n)$ extra.

Codigo. Ver `cpp.tex`, seccion *Closest pair of points*.

7.3.4 Rotating calipers (polygon diameter)

Idea intuitiva. Encuentra el **diametro** de un poligono convexo (par de vertices mas distantes) en $O(n)$. Asume poligono en orden counter-clockwise. Algoritmo: dos indices i (sobre las aristas) y k (sobre los vertices opuestos); en cada paso, rotar k mientras el area del paralelogramo aumenta. La distancia i - k puede ser maxima en cualquier momento del barrido.

Propiedades clave (invariantes).

- Cada indice avanza como maximo n veces total.
- El “diametro” es la maxima distancia entre dos vertices.
- Retorna **distancia al cuadrado**.

Complejidad.

- $O(n)$.

Donde tocar para modificar.

- **Reconstruir el par de vertices:** aniadir indices.
- **Anchura minima / maxima:** variantes de calipers.
- **Distancia minima entre poligonos convexos:** otra variante.

Trampas y errores frecuentes.

- Asume poligono **estrictamente** convexo (sin vertices colineales). Si hay colineales, aniadir perturbacion o filtrar.

Codigo. Ver `cpp.tex`, seccion *Rotating calipers (polygon diameter)*.

8 Miscelanea

8.1 Utilidades del usuario

8.1.1 Hora

Idea intuitiva. Una pequena clase para representar **tiempos en segundos** y hacer aritmetica basica sobre ellos. Internamente almacena todo en segundos (un solo `int`), lo cual simplifica comparaciones y diferencias. El constructor acepta (h, m, s) y los convierte a segundos. Hay un metodo `cut()` que fuerza un minimo (util para “horas trabajadas”).

Propiedades clave (invariantes).

- Representacion canonica: **solo segundos**. No hay zonas horarias, no hay DST.
- Comparadores `<` y `>` operan sobre los segundos.
- `operator-` retorna la **diferencia absoluta** en segundos (siempre no negativa).
- `cut()` aplica un piso (por defecto 8h 30m = 30600 segundos).

Complejidad.

- $O(1)$ por operacion.

Donde tocar para modificar.

- **Aniadir suma:** `operator+` que devuelve un `Hour` con la suma de segundos.
- **Aniadir output formateado:** `aniadir show_hms()` que imprime $H:M:S$.
- **Cambiar el “corte”:** modificar el valor magico 30’600 en `cut()` o pasarlo como parametro.
- **Tamano maximo:** si los tiempos exceden ~ 68 anos, el `int` desborda; usar `long long`.
- **Precision sub-segundo:** aniadir `ms` (milisegundos) y guardar en `long long` con factor 1000.

Trampas y errores frecuentes.

- El operador `-` retorna **valor absoluto**, no diferencia con signo. Si necesitas diferencia con signo, aniadir un nuevo metodo o sobrecargar.
- `cut()` modifica **s in-place**; si el llamador no espera esto, aniadir una version `cut(min)` que retorna un nuevo `Hour`.
- El umbral magico 30’600 (8h 30m) esta hardcoded; documentar bien que es “jornada minima”.

Codigo. Ver `cpp.tex`, seccion *Hora*.